United States Patent	12412034
Kind Code	B2
Date of Patent	September 09, 2025
Inventor(s)	John; Jae Min et al.

Transforming natural language to structured query language based on scalable search and content-based schema linking

Abstract

Techniques for preprocessing data assets to be used in a natural language to logical form model based on scalable search and content-based schema linking. In one particular aspect, a method includes accessing an utterance, classifying named entities within the utterance into predefined classes, searching value lists within the database schema using tokens from the utterance to identify and output value matches including: (i) any value within the value lists that matches a token from the utterance and (ii) any attribute associated with a matching value, generating a data structure by organizing and storing: (i) each of the named entities and an assigned class for each of the named entities, (ii) each of the value matches and the token matching each of the value matches, and (iii) the utterance, in a predefined format for the data structure, and outputting the data structure.

Inventors: **John; Jae Min (Redwood City, CA), Vishnoi; Vishal (Redwood City, CA), Johnson; Mark Edward (Castle Cove, AU), Duong; Thanh Long (Seabrook, AU), Gadde; Srinivasa Phani Kumar (Fremont, CA), Vinnakota; Balakota Srinivas (Sunnyvale, CA), Subramanian; Shivashankar (Melbourne, AU), Hoang; Cong Duy Vu (Melbourne, AU), Dharmasiri; Yakupitiyage Don Thanuja Samodhye (Melbourne, AU), Mathur; Nitika (Melbourne, AU), Kanuga; Aashna Devang (Foster City, CA), Arthur; Philip (Sydney, AU), Tangari; Gioacchino (Sydney, AU), Siu; Steve Wai-Chun (Melbourne, AU)**

Applicant: **Oracle International Corporation** (Redwood Shores, CA)

Family ID: **86694566**

Assignee: **Oracle International Corporation (Redwood Shores, CA)**

Appl. No.: **18/065387**

Filed: **December 13, 2022**

Prior Publication Data

Document Identifier	Publication Date

Related U.S. Application Data

us-provisional-application US 63361390 20211215
us-provisional-application US 63265392 20211214

Publication Classification

Int. Cl.: **G06F40/284** (20200101); **G06F16/242** (20190101); **G06F16/2452** (20190101); **G06F40/295** (20200101); **G06F40/40** (20200101); **G06F40/42** (20200101)

U.S. Cl.:

CPC **G06F40/284** (20200101); **G06F16/243** (20190101); **G06F16/24522** (20190101); **G06F40/295** (20200101); **G06F40/40** (20200101); **G06F40/42** (20200101);

Field of Classification Search

CPC: G06F (40/284); G06F (16/243); G06F (16/24522); G06F (40/42); G06F (40/40); G06F (40/295)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
10303683	12/2018	Anderson	N/A	G06F 16/285
11023461	12/2020	Rumiantsau	N/A	G06F 16/2246
11144726	12/2020	Chatterjee	N/A	G06N 3/08
11222013	12/2021	de Lima	N/A	G06F 40/295
11526507	12/2021	Zhong	N/A	G06F 16/24522
2018/0210883	12/2017	Ang	N/A	N/A
2020/0311349	12/2019	Balasubramanian	N/A	G06F 40/295
2022/0129450	12/2021	Cao	N/A	G06F 16/2433
2022/0318247	12/2021	Sen	N/A	G06F 16/243

OTHER PUBLICATIONS

Guo, Jiaqi, Zecheng Zhan, Yan Gao, Yan Xiao, Jian-Guang Lou, Ting Liu, and Dongmei Zhang, “Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation”, Jul. 2019, Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pp. 4524-4535. (Year: 2019). cited by examiner

Scholak, Torsten, Raymond Li, Dzmitry Bahdanau, Harm de Vries, and Christopher Pal, “DuoRAT: Towards Simpler Text-to-SQL Models”, Jun. 2021, Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pp. 1313-1321. (Year: 2021). cited by examiner

Deshmukh, Sulochana, and Marwan Bikdash, “Automatic Text-to-SQL Machine Translation for Scholarly Publication Database Search”, Mar. 2020, 2020 SoutheastCon, vol. 2, pp. 1-8. (Year: 2020). cited by examiner

Gormley, Clinton, and Zachary Tong, “Elasticsearch: The Definitive Guide: A Distributed Real-

Time Search and Analytics Engine", 2015, O'Reilly Media, Inc. (Year: 2015). cited by examiner
Wolfson, Tomer, Jonathan Berant, and Daniel Deutch, "Weakly Supervised Mapping of Natural Language to SQL through Question Decomposition", Dec. 12, 2021, arXiv preprint arXiv:2112.06311. (Year: 2021). cited by examiner
Saparina, Irina, and Anton Osokin, "SPARQLing Database Queries from Intermediate Question Decompositions", Nov. 2021, Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pp. 8984-8998. (Year: 2021). cited by examiner
Chen, Yi-Hui, Eric Jui-Lin Lu, and Ting-An Ou, "Intelligent SPARQL Query Generation for Natural Language Processing Systems", Nov. 2021, IEEE Access, vol. 9, pp. 158638-158650. (Year: 2021). cited by examiner
Brunner, Ursin, and Kurt Stockinger, "ValueNet: A Natural Language-to-SQL System that Learns from Database Information", Apr. 2021, 2021 IEEE 37th International Conference on Data Engineering (ICDE), pp. 2177-2182. (Year: 2021). cited by examiner
Lei, Wenqiang, Weixin Wang, Zhixin Ma, Tian Gan, Wei Lu, Min-Yen Kan, and Tat-Seng Chua, "Re-examining the Role of Schema Linking in Text-to-SQL", Nov. 2020, Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 6943-6954. (Year: 2020). cited by examiner
Montgomery, Chantal, Haruna Isah, and Farhana Zulkernine, "Towards a Natural Language Query Processing System", Sep. 2020, 2020 1st International Conference on Big Data Analytics and Practices (IBDAP), pp. 1-6. (Year: 2020). cited by examiner
Yu, Tao, Rui Zhang, Kai Yang, Michihiro Yasunaga, et al., "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task", Oct. 2018, Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pp. 3911-3921. (Year: 2018). cited by examiner
Cai, Ruichu, Boyan Xu, Zhenjie Zhang, Xiaoyan Yang, Zijian Li, and Zhihao Liang, "An Encoder-Decoder Framework Translating Natural Language to Database Queries", Jul. 2018, Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI-18), pp. 3977-3983. (Year: 2018). cited by examiner
Gormley et al., Elasticsearch: The Definitive Guide, O'Reilly Media, Inc., Available online at http://stmarysguntur.com/wp-content/uploads/2019/04/1021302647.pdf, Jan. 2015, 719 pages. cited by applicant
Scholak et al., DuoRAT: Towards Simpler Text-to-SQL Models, Available online at https://aclanthology.org/2021.naacl-main.103.pdf, Sep. 10, 2021, 9 pages. cited by applicant
Wang et al., RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers, Available online at https://arxiv.org/pdf/1911.04942.pdf, Aug. 24, 2021, 12 pages. cited by applicant

---

Primary Examiner: Washburn; Daniel C

Assistant Examiner: Boggs; James

Attorney, Agent or Firm: Kilpatrick Townsend & Stockton LLP

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS (1) The present application is a non-provisional application of and claims benefit and priority under 35 U.S.C. 119(e) of U.S. Provisional Application No. 63/265,392 having a filing date of Dec. 14, 2021 and U.S. Provisional

Application No. 63/361,390 having a filing date of Dec. 15, 2021, the entire contents of which are incorporated herein by reference for all purposes.

FIELD

(1) The present disclosure relates generally to transforming natural language to Structured Query Language (SQL), and more particularly, to techniques for preprocessing data assets to be used in a natural language to logical form model (e.g., a natural language to SQL (NL2SQL) model) based on scalable search and content-based schema linking.

BACKGROUND

(2) Structured Query Language (SQL) is a domain-specific language used in programming and designed for managing data held in a relational database management system (RDBMS), or for stream processing in a relational data stream management system (RDSMS). It is particularly useful in handling structured data (i.e., data incorporating relations among entities and variables). SQL includes sublanguages such as a data query language (DQL), a data definition language (DDL), a data control language (DCL), and a data manipulation language (DML). The scope of SQL includes data query, data manipulation (insert, update, and delete), data definition (schema creation and modification), and data access control. Although SQL is essentially a declarative language (4GL), it also includes procedural elements. In order to effectively leverage data, RDBMS and RDSMS users are required to not only have prior knowledge about the database schema (e.g., table and column names) but also a working understanding of the syntax and semantics of SQL. Nonetheless, despite its expressiveness, SQL can often hinder non-technical users from exploring and making use of their data.

(3) Natural language is an alternative interface to data held or implemented in RDBMS and RDSMS because it allows non-technical users to formulate complex questions in a more concise manner than SQL. Using semantic parsing, natural language statements, requests, and questions can be transformed into logical forms or meaning representations that can be executed by an application (e.g., model, program, machine, etc.). For example, semantic parsing can transform natural language sentences directly into general purpose programming languages such as Python, Java, and SQL. Processes for transforming natural language sentences to SQL queries typically include rule-based, statistical-based, and deep learning-based systems. Rule-based systems typically use a series of fixed rules to translate the natural language sentences to SQL queries. Rule-based systems are generally domain-specific and, thus, are considered inelastic and do not generalize well to new use cases (e.g., across different domains). Statistical-based systems label tokens (i.e., words or phrases) in an input natural language sentence according to their semantic role in the sentence and use the labels to fill slots in the SQL query but have limitations on the types of sentences that can be parsed (e.g., a sentence must be able to be represented as a parse tree). Deep-learning based systems, such as sequence-to-sequence models, involve training deep-learning models that directly translate the natural language sentences to SQL queries and have been shown to generalize well to new use cases.

BRIEF SUMMARY

(4) Techniques are disclosed for preprocessing data assets to be used in a natural language to logical form model (e.g., an NL2SQL model) based on scalable search and content-based schema linking.

(5) In some embodiments, a computer-implemented method includes accessing an utterance; classifying one or more named entities within the utterance into predefined classes, wherein the classifying results in each of the one or more named entities being assigned a class from the predefined classes, and the predefined classes match table names or attribute names within a database schema; searching one or more value lists within the database schema using one or more tokens from the utterance, wherein the searching identifies and outputs one or more value matches comprising: (i) any value within the one or more value lists that matches a token from the utterance

and (ii) any attribute associated with a matching value; generating a data structure, according to a predefined format for the data structure, by storing: (i) each of the one or more named entities and the assigned class for each of the one or more named entities, (ii) each of the value matches and the token matching each of the value matches, and (iii) the utterance; and outputting the data structure.
(6) In some embodiments, the method further includes extracting information from the database schema for each of the one or more named entities based on the class assigned to each of the one or more named entities, wherein the information includes a predefined format for values within a table or attribute associated with the class via the matching table name or attribute name, and wherein the generating the data structure comprises formatting each of the one or more named entities in the predefined format associated with the class assigned to each of the one or more named entities.
(7) In some embodiments, the classifying extracts matched offsets for each of the one or more named entities, which are a numerical beginning and an ending position of the each of the one or more named entities within the utterance, and wherein the generating the data structure further comprises organizing and storing the matched offsets for each of the one or more named entities in the predefined format for the data structure.
(8) In some embodiments, the one or more value lists are structured within one or more indexes, and the search is performed by running queries on the one or more indexes using the tokens from the utterance.
(9) In some embodiments, the searching extracts matched offsets for each of the tokens from the utterance, which are a numerical beginning and an ending position of the each of the tokens in the utterance, and wherein the generating the data structure further comprises organizing and storing the matched offsets for each of the tokens in the predefined format for the data structure.
(10) In some embodiments, the one or more value lists comprise one or more attributes, values, and synonyms of the values, and the searching identifies and outputs value matches comprising: (i) any value within the one or more value lists that matches a token from the utterance, (ii) any value within the one or more value lists that has a synonym of the value that matches a token from the utterance, and (iii) any attribute associated with a matching value.
(11) In some embodiments, the method further includes inputting the data structure into a machine learning model; translating, using the machine learning model, the utterance into a logical form based on the data structure; executing the logical form as a query on a database associated with the database schema to retrieve a result for the query; and outputting the result for the utterance.
(12) Some embodiments include a system that includes one or more data processors and one or more non-transitory computer-readable media storing instructions which, when executed by the one or more data processors, cause the one or more data processors to perform part or all of the operations and/or methods disclosed herein.
(13) Some embodiments include a computer-program product tangibly embodied in one or more non-transitory machine-readable media, including instructions configured to cause one or more data processors to perform part or all of the operations and/or methods disclosed herein.
(14) The techniques described above and below may be implemented in a number of ways and in a number of contexts. Several example implementations and contexts are provided with reference to the following figures, as described below in more detail. However, the following implementations and contexts are but a few of many.

Description

BRIEF DESCRIPTION OF THE DRAWINGS
(1) FIG. **1** is a simplified block diagram of a distributed environment incorporating an exemplary embodiment.
(2) FIG. **2** is a simplified block diagram of a computing system implementing a master bot

according to various embodiments.
(3) FIG. **3** is a simplified block diagram of a computing system implementing a skill bot according to various embodiments.
(4) FIG. **4**A is a simplified block diagram of a scalable search and content-based schema linking architecture according to various embodiments.
(5) FIG. **4**B is an exemplary user interface showing that value list names do not have to match with attribute names according to various embodiments.
(6) FIG. **5** illustrates an example process for preprocessing utterances based on scalable search and content-based schema linking according to various embodiments.
(7) FIG. **6** depicts a simplified diagram of a distributed system for implementing various embodiments.
(8) FIG. **7** is a simplified block diagram of one or more components of a system environment by which services provided by one or more components of an embodiment system may be offered as cloud services, in accordance with various embodiments.
(9) FIG. **8** illustrates an example computer system that may be used to implement various embodiments.

DETAILED DESCRIPTION

(10) In the following description, for the purposes of explanation, specific details are set forth in order to provide a thorough understanding of certain embodiments. However, it will be apparent that various embodiments may be practiced without these specific details. The figures and description are not intended to be restrictive. The word “exemplary” is used herein to mean “serving as an example, instance, or illustration.” Any embodiment or design described herein as “exemplary” is not necessarily to be construed as preferred or advantageous over other embodiments or designs.

Introduction

(11) In recent years, the amount of data powering different industries, and their systems has been increasing exponentially. The majority of business information is managed by relational databases that store, process, and retrieve data. Databases power information systems across multiple industries including retail (e.g., orders, cancellations, refunds), supply chain (e.g., raw materials, stocks, vendors), healthcare (e.g., medical records), and finance (e.g., financial business metrics) to name a few. Additionally, databases power customer support mechanisms, Internet search engines and knowledge bases, and much more. It is imperative for modern data-driven companies to track, in real-time, the states of their companies and their businesses in order to quickly understand and diagnose any emerging issues, trends, or anomalies and take corrective actions. This tracking is usually performed manually by business analysts interfacing with databases using complex queries in declarative query languages like Structured Query Language (SQL).
(12) Although SQL queries that address fundamental business metrics are common, predefined, and incorporated in commercial products that power insights into business metrics, other non-fundamental business metrics or follow-up business metrics must be manually coded by the analysts. Such static interactions between database queries and consumption of the corresponding results require time-consuming manual intervention and result in slow feedback cycles. It is vastly more efficient to have non-technical business leaders directly interact with the analytics tables via natural language queries that abstract away the underlying SQL code. Defining a SQL query requires a strong understanding of database schema and SQL syntax and can quickly get overwhelming for beginners and non-technical stakeholders. Efforts to bridge this communication gap have led to the development of a new type of processing called Natural Language Interface to Database (NLIDB). NLIDB allows users to access database information using natural language inquiries. This natural language database search capability has become more popular over recent years and, as such, companies are developing deep-learning approaches for accessing specific databases using natural language. One such approach is NL2SQL. NL2SQL seeks to transform

natural language statements, requests, and questions (i.e., sentences) to SQL queries so that individuals, including those unfamiliar with SQL, can run unstructured queries against databases. Additionally, the SQL queries can enable digital assistants, such as chatbots, to improve their responses when an answer or response to a query can be found in different databases with different schema.

(13) In order to perform well, an NL2SQL translator including an NL2SQL model should be able to accurately correlate entities in the user's natural language query with entities in the database the user is querying. Conventional NL2SQL translators generally rely on schema linking in which features extracted from the user's natural language query are matched to features of the database's schema (e.g., tables, columns, values). In many cases, due to the diversity and ambiguity of mentions in the user's natural language query, the extracted features do not exactly match the features of the database's schema and the NL2SQL translator fails to provide satisfactory responses to the user's natural language query even though the extracted features relate to, are derived from, and/or are similar to the features of the database's schema. Conventional approaches have attempted to solve this problem by relying on a SQL similarity operator such as the SQL LIKE operator, but this approach has often led to reduced NL2SQL translator performance and slow responses to the user's natural language query.

(14) To overcome these challenges and others, techniques for preprocessing data assets to be used in a natural language to logical form model (e.g., an NL2SQL model) based on scalable search and content-based schema linking are provided herein. The preprocessing techniques combine named entity recognition and scalable searches (e.g., Elasticsearch) to obtain content-based schema linking (CBSL) matches between words or tokens of an utterance and system entities and/or values for attributes within a given database schema. The content-based schema linking matches are appended to the utterance using a unique data structure, and then the data structure is input into the natural language to logical form model. The data structure facilitates encoding and decoding of the input utterance into a logical form by the natural language to logical form model. Advantageously, by using scalable search and content-based schema linking as described herein, the natural language to logical form model response time and accuracy can be improved over conventional standardized programming language techniques such as SQL searches on the database using a SQL similarity operator.

(15) In one particular aspect, a method includes accessing an utterance; classifying one or more named entities within the utterance into predefined classes, wherein the classifying results in each of the one or more named entities being assigned a class from the predefined classes, and the predefined classes match table names or attribute names within a database schema; searching one or more value lists within the database schema using one or more tokens from the utterance, wherein the searching identifies and outputs one or more value matches comprising: (i) any value within the one or more value lists that matches a token from the utterance and (ii) any attribute associated with a matching value; generating a data structure, according to a predefined format for the data structure, by storing: (i) each of the one or more named entities and the assigned class for each of the one or more named entities, (ii) each of the value matches and the token matching each of the value matches, and (iii) the utterance; and outputting the data structure.

(16) Bot and Analytic Systems

(17) A bot (also referred to as a skill, chatbot, chatterbot, or talkbot) is a computer program that can perform conversations with end users. The bot can generally respond to natural-language messages (e.g., questions or comments) through a messaging application that uses natural-language messages. Enterprises may use one or more bots to communicate with end users through a messaging application. The messaging application may include, for example, over-the-top (OTT) messaging channels (such as Facebook Messenger, Facebook WhatsApp, WeChat, Line, Kik, Telegram, Talk, Skype, Slack, or SMS), virtual private assistants (such as Amazon Dot, Echo, or Show, Google Home, Apple HomePod, etc.), mobile and web app extensions that extend native or

hybrid/responsive mobile apps or web applications with chat capabilities, or voice based input (such as devices or apps with interfaces that use Siri, Cortana, Google Voice, or other speech input for interaction).

(18) In some examples, the bot may be associated with a Uniform Resource Identifier (URI). The URI may identify the bot using a string of characters. The URI may be used as a webhook for one or more messaging application systems. The URI may include, for example, a Uniform Resource Locator (URL) or a Uniform Resource Name (URN). The bot may be designed to receive a message (e.g., a hypertext transfer protocol (HTTP) post call message) from a messaging application system. The HTTP post call message may be directed to the URI from the messaging application system. In some examples, the message may be different from a HTTP post call message. For example, the bot may receive a message from a Short Message Service (SMS). While discussion herein refers to communications that the bot receives as a message, it should be understood that the message may be an HTTP post call message, a SMS message, or any other type of communication between two systems.

(19) End users interact with the bot through conversational interactions (sometimes referred to as a conversational user interface (UI)), just as end users interact with other people. In some cases, the conversational interactions may include the end user saying “Hello” to the bot and the bot responding with a “Hi” and asking the end user how it can help. End users also interact with the bot through other types of interactions, such as transactional interactions (e.g., with a banking bot that is at least trained to transfer money from one account to another), informational interactions (e.g., with a human resources bot that is at least trained check the remaining vacation hours the user has), and/or retail interactions (e.g., with a retail bot that is at least trained for discussing returning purchased goods or seeking technical support).

(20) In some examples, the bot may intelligently handle end user interactions without intervention by an administrator or developer of the bot. For example, an end user may send one or more messages to the bot in order to achieve a desired goal. A message may include certain content, such as text, emojis, audio, image, video, or other method of conveying a message. In some examples, the bot may automatically convert content into a standardized form and generate a natural language response. The bot may also automatically prompt the end user for additional input parameters or request other additional information. In some examples, the bot may also initiate communication with the end user, rather than passively responding to end user utterances.

(21) A conversation with a bot may follow a specific conversation flow including multiple states. The flow may define what would happen next based on an input. In some examples, a state machine that includes user defined states (e.g., end user intents) and actions to take in the states or from state to state may be used to implement the bot. A conversation may take different paths based on the end user input, which may impact the decision the bot makes for the flow. For example, at each state, based on the end user input or utterances, the bot may determine the end user's intent in order to determine the appropriate next action to take. As used herein and in the context of an utterance, the term “intent” refers to an intent of the user who provided the utterance. For example, the user may intend to engage the bot in a conversation to order pizza, where the user's intent would be represented through the utterance “order pizza.” A user intent can be directed to a particular task that the user wishes the bot to perform on behalf of the user. Therefore, utterances reflecting the user's intent can be phrased as questions, commands, requests, and the like.

(22) In the context of the configuration of the bot, the term “intent” is also used herein to refer to configuration information for mapping a user's utterance to a specific task/action or category of task/action that the bot can perform. In order to distinguish between the intent of an utterance (i.e., a user intent) and the intent of the bot, the latter is sometimes referred to herein as a “bot intent.” A bot intent may comprise a set of one or more utterances associated with the intent. For instance, an intent for ordering pizza can have various permutations of utterances that express a desire to place an order for pizza. These associated utterances can be used to train an intent classifier of the bot to

enable the intent classifier to subsequently determine whether an input utterance from a user matches the order pizza intent. Bot intents may be associated with one or more dialog flows for starting a conversation with the user and in a certain state. For example, the first message for the order pizza intent could be the question “What kind of pizza would you like?” In addition to associated utterances, bot intents may further comprise named entities that relate to the intent. For example, the order pizza intent could include variables or parameters used to perform the task of ordering pizza (e.g., topping **1**, topping **2**, pizza type, pizza size, pizza quantity, and the like). The value of an entity is typically obtained through conversing with the user.

(23) FIG. **1** is a simplified block diagram of an environment **100** incorporating a chatbot system according to certain embodiments. Environment **100** comprises a digital assistant builder platform (DABP) **102** that enables users **104** of DABP **102** to create and deploy digital assistants or chatbot systems. DABP **102** can be used to create one or more digital assistants (or DAs) or chatbot systems. For example, as shown in FIG. **1**, users **104** representing a particular enterprise can use DABP **102** to create and deploy a digital assistant **106** for users of the particular enterprise. For example, DABP **102** can be used by a bank to create one or more digital assistants for use by the bank's customers. The same DABP **102** platform can be used by multiple enterprises to create digital assistants. As another example, an owner of a restaurant (e.g., a pizza shop) may use DABP **102** to create and deploy a digital assistant that enables customers of the restaurant to order food (e.g., order pizza).

(24) For purposes of this disclosure, a “digital assistant” is a tool that helps users of the digital assistant accomplish various tasks through natural language conversations. A digital assistant can be implemented using software only (e.g., the digital assistant is a digital tool implemented using programs, code, or instructions executable by one or more processors), using hardware, or using a combination of hardware and software. A digital assistant can be embodied or implemented in various physical systems or devices, such as in a computer, a mobile phone, a watch, an appliance, a vehicle, and the like. A digital assistant is also sometimes referred to as a chatbot system. Accordingly, for purposes of this disclosure, the terms digital assistant and chatbot system are interchangeable.

(25) A digital assistant, such as digital assistant **106** built using DABP **102**, can be used to perform various tasks via natural language-based conversations between the digital assistant and its users **108**. As part of a conversation, a user may provide one or more user inputs **110** to digital assistant **106** and get responses **112** back from digital assistant **106**. A conversation can include one or more of inputs **110** and responses **112**. Via these conversations, a user can request one or more tasks to be performed by the digital assistant and, in response, the digital assistant is configured to perform the user-requested tasks and respond with appropriate responses to the user.

(26) User inputs **110** are generally in a natural language form and are referred to as utterances. A user utterance **110** can be in text form, such as when a user types in a sentence, a question, a text fragment, or even a single word and provides it as input to digital assistant **106**. In some examples, a user utterance **110** can be in audio input or speech form, such as when a user says or speaks something that is provided as input to digital assistant **106**. The utterances are typically in a language spoken by the user. For example, the utterances may be in English, or some other language. When an utterance is in speech form, the speech input is converted to text form utterances in that particular language and the text utterances are then processed by digital assistant **106**. Various speech-to-text processing techniques may be used to convert a speech or audio input to a text utterance, which is then processed by digital assistant **106**. In some examples, the speech-to-text conversion may be done by digital assistant **106** itself.

(27) An utterance, which may be a text utterance or a speech utterance, can be a fragment, a sentence, multiple sentences, one or more words, one or more questions, combinations of the aforementioned types, and the like. Digital assistant **106** is configured to apply natural language understanding (NLU) techniques to the utterance to understand the meaning of the user input. As

part of the NLU processing for an utterance, digital assistant **106** is configured to perform processing to understand the meaning of the utterance, which involves identifying one or more intents and one or more entities corresponding to the utterance. Upon understanding the meaning of an utterance, digital assistant **106** may perform one or more actions or operations responsive to the understood meaning or intents. For purposes of this disclosure, it is assumed that the utterances are text utterances that have been provided directly by a user of digital assistant **106** or are the results of conversion of input speech utterances to text form. This however is not intended to be limiting or restrictive in any manner.

(28) For example, a user input may request a pizza to be ordered by providing an utterance such as "I want to order a pizza." Upon receiving such an utterance, digital assistant **106** is configured to understand the meaning of the utterance and take appropriate actions. The appropriate actions may involve, for example, responding to the user with questions requesting user input on the type of pizza the user desires to order, the size of the pizza, any toppings for the pizza, and the like. The responses provided by digital assistant **106** may also be in natural language form and typically in the same language as the input utterance. As part of generating these responses, digital assistant **106** may perform natural language generation (NLG). For the user ordering a pizza, via the conversation between the user and digital assistant **106**, the digital assistant may guide the user to provide all the requisite information for the pizza order, and then at the end of the conversation cause the pizza to be ordered. Digital assistant **106** may end the conversation by outputting information to the user indicating that the pizza has been ordered.

(29) At a conceptual level, digital assistant **106** performs various processing in response to an utterance received from a user. In some examples, this processing involves a series or pipeline of processing steps including, for example, understanding the meaning of the input utterance, determining an action to be performed in response to the utterance, where appropriate causing the action to be performed, generating a response to be output to the user responsive to the user utterance, outputting the response to the user, and the like. The NLU processing can include parsing the received input utterance to understand the structure and meaning of the utterance, refining and reforming the utterance to develop a better understandable form (e.g., logical form) or structure for the utterance. Generating a response may include using NLG techniques.

(30) The NLU processing performed by a digital assistant, such as digital assistant **106**, can include various NLP related tasks such as sentence parsing (e.g., tokenizing, lemmatizing, identifying part-of-speech tags for the sentence, identifying named entities in the sentence, generating dependency trees to represent the sentence structure, splitting a sentence into clauses, analyzing individual clauses, resolving anaphoras, performing chunking, and the like). In certain examples, the NLU processing is performed by digital assistant **106** itself. In some other examples, digital assistant **106** may use other resources to perform portions of the NLU processing. For example, the syntax and structure of an input utterance sentence may be identified by processing the sentence using a parser, a part-of-speech tagger, and/or a NER. In one implementation, for the English language, a parser, a part-of-speech tagger, and a named entity recognizer such as ones provided by the Stanford NLP Group are used for analyzing the sentence structure and syntax. These are provided as part of the Stanford CoreNLP toolkit.

(31) While the various examples provided in this disclosure show utterances in the English language, this is meant only as an example. In certain examples, digital assistant **106** is also capable of handling utterances in languages other than English. Digital assistant **106** may provide subsystems (e.g., components implementing NLU functionality) that are configured for performing processing for different languages. These subsystems may be implemented as pluggable units that can be called using service calls from an NLU core server. This makes the NLU processing flexible and extensible for each language, including allowing different orders of processing. A language pack may be provided for individual languages, where a language pack can register a list of subsystems that can be served from the NLU core server.

(32) A digital assistant, such as digital assistant **106** depicted in FIG. **1**, can be made available or accessible to its users **108** through a variety of different channels, such as but not limited to, via certain applications, via social media platforms, via various messaging services and applications, and other applications or channels. A single digital assistant can have several channels configured for it so that it can be run on and be accessed by different services simultaneously.
(33) A digital assistant or chatbot system generally contains or is associated with one or more skills. In certain embodiments, these skills are individual chatbots (referred to as skill bots) that are configured to interact with users and fulfill specific types of tasks, such as tracking inventory, submitting timecards, creating expense reports, ordering food, checking a bank account, making reservations, buying a widget, and the like. For example, for the embodiment depicted in FIG. **1**, digital assistant or chatbot system **106** includes skills **116-1**, **116-2**, **116-3**, and so on. For purposes of this disclosure, the terms "skill" and "skills" are used synonymously with the terms "skill bot" and "skill bots," respectively.
(34) Each skill associated with a digital assistant helps a user of the digital assistant complete a task through a conversation with the user, where the conversation can include a combination of text or audio inputs provided by the user and responses provided by the skill bots. These responses may be in the form of text or audio messages to the user and/or using simple user interface elements (e.g., select lists) that are presented to the user for the user to make selections.
(35) There are various ways in which a skill or skill bot can be associated or added to a digital assistant. In some instances, a skill bot can be developed by an enterprise and then added to a digital assistant using DABP **102**. In other instances, a skill bot can be developed and created using DABP **102** and then added to a digital assistant created using DABP **102**. In yet other instances, DABP **102** provides an online digital store (referred to as a "skills store") that offers multiple skills directed to a wide range of tasks. The skills offered through the skills store may also expose various cloud services. In order to add a skill to a digital assistant being generated using DABP **102**, a user of DABP **102** can access the skills store via DABP **102**, select a desired skill, and indicate that the selected skill is to be added to the digital assistant created using DABP **102**. A skill from the skills store can be added to a digital assistant as is or in a modified form (for example, a user of DABP **102** may select and clone a particular skill bot provided by the skills store, make customizations or modifications to the selected skill bot, and then add the modified skill bot to a digital assistant created using DABP **102**).
(36) Various different architectures may be used to implement a digital assistant or chatbot system. For example, in certain embodiments, the digital assistants created and deployed using DABP **102** may be implemented using a master bot/child (or sub) bot paradigm or architecture. According to this paradigm, a digital assistant is implemented as a master bot that interacts with one or more child bots that are skill bots. For example, in the embodiment depicted in FIG. **1**, digital assistant **106** comprises a master bot **114** and skill bots **116-1**, **116-2**, etc. that are child bots of master bot **114**. In certain examples, digital assistant **106** is itself considered to act as the master bot.
(37) A digital assistant implemented according to the master-child bot architecture enables users of the digital assistant to interact with multiple skills through a unified user interface, namely via the master bot. When a user engages with a digital assistant, the user input is received by the master bot. The master bot then performs processing to determine the meaning of the user input utterance. The master bot then determines whether the task requested by the user in the utterance can be handled by the master bot itself, else the master bot selects an appropriate skill bot for handling the user request and routes the conversation to the selected skill bot. This enables a user to converse with the digital assistant through a common single interface and still provide the capability to use several skill bots configured to perform specific tasks. For example, for a digital assistance developed for an enterprise, the master bot of the digital assistant may interface with skill bots with specific functionalities, such as a customer relationship management (CRM) bot for performing functions related to customer relationship management, an enterprise resource planning (ERP) bot

for performing functions related to enterprise resource planning, a human capital management (HCM) bot for performing functions related to human capital management, etc. This way the end user or consumer of the digital assistant need only know how to access the digital assistant through the common master bot interface and behind the scenes multiple skill bots are provided for handling the user request.

(38) In certain examples, in a master bot/child bots infrastructure, the master bot is configured to be aware of the available list of skill bots. The master bot may have access to metadata that identifies the various available skill bots, and for each skill bot, the capabilities of the skill bot including the tasks that can be performed by the skill bot. Upon receiving a user request in the form of an utterance, the master bot is configured to, from the multiple available skill bots, identify or predict a specific skill bot that can best serve or handle the user request. The master bot then routes the utterance (or a portion of the utterance) to that specific skill bot for further handling. Control thus flows from the master bot to the skill bots. The master bot can support multiple input and output channels. In certain examples, routing may be performed with the aid of processing performed by one or more available skill bots. For example, as discussed below, a skill bot can be trained to infer an intent for an utterance and to determine whether the inferred intent matches an intent with which the skill bot is configured. Thus, the routing performed by the master bot can involve the skill bot communicating to the master bot an indication of whether the skill bot has been configured with an intent suitable for handling the utterance.

(39) While the embodiment in FIG. **1** shows digital assistant **106** comprising a master bot **114** and skill bots **116-1**, **116-2**, and **116-3**, this is not intended to be limiting. A digital assistant can include various other components (e.g., other systems and subsystems) that provide the functionalities of the digital assistant. These systems and subsystems may be implemented only in software (e.g., code, instructions stored on a computer-readable medium and executable by one or more processors), in hardware only, or in implementations that use a combination of software and hardware.

(40) DABP **102** provides an infrastructure and various services and features that enable a user of DABP **102** to create a digital assistant including one or more skill bots associated with the digital assistant. In some instances, a skill bot can be created by cloning an existing skill bot, for example, cloning a skill bot provided by the skills store. As previously indicated, DABP **102** provides a skills store or skills catalog that offers multiple skill bots for performing various tasks. A user of DABP **102** can clone a skill bot from the skills store. As needed, modifications or customizations may be made to the cloned skill bot. In some other instances, a user of DABP **102** created a skill bot from scratch using tools and services offered by DABP **102**. As previously indicated, the skills store or skills catalog provided by DABP **102** may offer multiple skill bots for performing various tasks.

(41) In certain examples, at a high level, creating or customizing a skill bot involves the following steps: (1) Configuring settings for a new skill bot (2) Configuring one or more intents for the skill bot (3) Configuring one or more entities for one or more intents (4) Training the skill bot (5) Creating a dialog flow for the skill bot (6) Adding custom components to the skill bot as needed (7) Testing and deploying the skill bot

Each of the above steps is briefly described below. (1) Configuring settings for a new skill bot—Various settings may be configured for the skill bot. For example, a skill bot designer can specify one or more invocation names for the skill bot being created. These invocation names can then be used by users of a digital assistant to explicitly invoke the skill bot. For example, a user can input an invocation name in the user's utterance to explicitly invoke the corresponding skill bot. (2) Configuring one or more intents and associated example utterances for the skill bot—The skill bot designer specifies one or more intents (also referred to as bot intents) for a skill bot being created. The skill bot is then trained based upon these specified intents. These intents represent categories or classes that the skill bot is trained to infer for input utterances. Upon receiving an utterance, a trained skill bot infers an intent for the utterance, where the inferred intent is selected from the

predefined set of intents used to train the skill bot. The skill bot then takes an appropriate action responsive to an utterance based upon the intent inferred for that utterance. In some instances, the intents for a skill bot represent tasks that the skill bot can perform for users of the digital assistant. Each intent is given an intent identifier or intent name. For example, for a skill bot trained for a bank, the intents specified for the skill bot may include “CheckBalance,” “TransferMoney,” “DepositCheck,” and the like.

(42) For each intent defined for a skill bot, the skill bot designer may also provide one or more example utterances that are representative of and illustrate the intent. These example utterances are meant to represent utterances that a user may input to the skill bot for that intent. For example, for the CheckBalance intent, example utterances may include “What's my savings account balance?”, “How much is in my checking account?”, “How much money do I have in my account,” and the like. Accordingly, various permutations of typical user utterances may be specified as example utterances for an intent.

(43) The intents and their associated example utterances are used as training data to train the skill bot. Various different training techniques may be used. As a result of this training, a predictive model is generated that is configured to take an utterance as input and output an intent inferred for the utterance by the predictive model. In some instances, input utterances are provided to an intent analysis engine, which is configured to use the trained model to predict or infer an intent for the input utterance. The skill bot may then take one or more actions based upon the inferred intent. (3) Configuring entities for one or more intents of the skill bot—In some instances, additional context may be needed to enable the skill bot to properly respond to a user utterance. For example, there may be situations where a user input utterance resolves to the same intent in a skill bot. For instance, in the above example, utterances “What's my savings account balance?” and “How much is in my checking account?” both resolve to the same CheckBalance intent, but these utterances are different requests asking for different things. To clarify such requests, one or more entities are added to an intent. Using the banking skill bot example, an entity called AccountType, which defines values called “checking” and “saving” may enable the skill bot to parse the user request and respond appropriately. In the above example, while the utterances resolve to the same intent, the value associated with the AccountType entity is different for the two utterances. This enables the skill bot to perform possibly different actions for the two utterances in spite of them resolving to the same intent. One or more entities can be specified for certain intents configured for the skill bot. Entities are thus used to add context to the intent itself. Entities help describe an intent more fully and enable the skill bot to complete a user request.

(44) In certain examples, there are two types of entities: (a) built-in entities provided by DABP **102**, and (2) custom entities that can be specified by a skill bot designer. Built-in entities are generic entities that can be used with a wide variety of bots. Examples of built-in entities include, without limitation, entities related to time, date, addresses, numbers, email addresses, duration, recurring time periods, currencies, phone numbers, URLs, and the like. Custom entities are used for more customized applications. For example, for a banking skill, an AccountType entity may be defined by the skill bot designer that enables various banking transactions by checking the user input for keywords like checking, savings, and credit cards, etc. (4) Training the skill bot—A skill bot is configured to receive user input in the form of utterances parse or otherwise process the received input and identify or select an intent that is relevant to the received user input. As indicated above, the skill bot has to be trained for this. In certain embodiments, a skill bot is trained based upon the intents configured for the skill bot and the example utterances associated with the intents (collectively, the training data), so that the skill bot can resolve user input utterances to one of its configured intents. In certain examples, the skill bot uses a predictive model that is trained using the training data and allows the skill bot to discern what users say (or in some cases, are trying to say). DABP **102** provides various different training techniques that can be used by a skill bot designer to train a skill bot, including various machine-learning based training techniques,

rules-based training techniques, and/or combinations thereof. In certain examples, a portion (e.g., 80%) of the training data is used to train a skill bot model and another portion (e.g., the remaining 20%) is used to test or verify the model. Once trained, the trained model (also sometimes referred to as the trained skill bot) can then be used to handle and respond to user utterances. In certain cases, a user's utterance may be a question that requires only a single answer and no further conversation. In order to handle such situations, a Q&A (question-and-answer) intent may be defined for a skill bot. This enables a skill bot to output replies to user requests without having to update the dialog definition. Q&A intents are created in a similar manner as regular intents. The dialog flow for Q&A intents can be different from that for regular intents. (5) Creating a dialog flow for the skill bot—A dialog flow specified for a skill bot describes how the skill bot reacts as different intents for the skill bot are resolved responsive to received user input. The dialog flow defines operations or actions that a skill bot will take, e.g., how the skill bot responds to user utterances, how the skill bot prompts users for input, how the skill bot returns data. A dialog flow is like a flowchart that is followed by the skill bot. The skill bot designer specifies a dialog flow using a language, such as markdown language. In certain embodiments, a version of YAML called OBotML may be used to specify a dialog flow for a skill bot. The dialog flow definition for a skill bot acts as a model for the conversation itself, one that lets the skill bot designer choreograph the interactions between a skill bot and the users that the skill bot services.

(45) In certain examples, the dialog flow definition for a skill bot contains three sections: (a) a context section (b) a default transitions section (c) a states section

(46) Context section—The skill bot designer can define variables that are used in a conversation flow in the context section. Other variables that may be named in the context section include, without limitation: variables for error handling, variables for built-in or custom entities, user variables that enable the skill bot to recognize and persist user preferences, and the like.

(47) Default transitions section—Transitions for a skill bot can be defined in the dialog flow states section or in the default transitions section. The transitions defined in the default transition section act as a fallback and get triggered when there are no applicable transitions defined within a state, or the conditions required to trigger a state transition cannot be met. The default transitions section can be used to define routing that allows the skill bot to gracefully handle unexpected user actions.

(48) States section—A dialog flow and its related operations are defined as a sequence of transitory states, which manage the logic within the dialog flow. Each state node within a dialog flow definition names a component that provides the functionality needed at that point in the dialog. States are thus built around the components. A state contains component-specific properties and defines the transitions to other states that get triggered after the component executes.

(49) Special case scenarios may be handled using the states sections. For example, there might be times when you want to provide users the option to temporarily leave a first skill, they are engaged with to do something in a second skill within the digital assistant. For example, if a user is engaged in a conversation with a shopping skill (e.g., the user has made some selections for purchase), the user may want to jump to a banking skill (e.g., the user may want to ensure that he/she has enough money for the purchase), and then return to the shopping skill to complete the user's order. To address this, an action in the first skill can be configured to initiate an interaction with the second different skill in the same digital assistant and then return to the original flow. (6) Adding custom components to the skill bot—As described above, states specified in a dialog flow for skill bot name components that provide the functionality needed corresponding to the states. Components enable a skill bot to perform functions. In certain embodiments, DABP **102** provides a set of preconfigured components for performing a wide range of functions. A skill bot designer can select one of more of these preconfigured components and associate them with states in the dialog flow for a skill bot. The skill bot designer can also create custom or new components using tools provided by DABP **102** and associate the custom components with one or more states in the dialog flow for a skill bot. (7) Testing and deploying the skill bot—DABP **102** provides several features

that enable the skill bot designer to test a skill bot being developed. The skill bot can then be deployed and included in a digital assistant.

(50) While the description above describes how to create a skill bot, similar techniques may also be used to create a digital assistant (or the master bot). At the master bot or digital assistant level, built-in system intents may be configured for the digital assistant. These built-in system intents are used to identify general tasks that the digital assistant itself (i.e., the master bot) can handle without invoking a skill bot associated with the digital assistant. Examples of system intents defined for a master bot include: (1) Exit: applies when the user signals the desire to exit the current conversation or context in the digital assistant; (2) Help: applies when the user asks for help or orientation; and (3) Unresolved Intent: applies to user input that doesn't match well with the exit and help intents. The digital assistant also stores information about the one or more skill bots associated with the digital assistant. This information enables the master bot to select a particular skill bot for handling an utterance.

(51) At the master bot or digital assistant level, when a user inputs a phrase or utterance to the digital assistant, the digital assistant is configured to perform processing to determine how to route the utterance and the related conversation. The digital assistant determines this using a routing model, which can be rules-based, AI-based, or a combination thereof. The digital assistant uses the routing model to determine whether the conversation corresponding to the user input utterance is to be routed to a particular skill for handling, is to be handled by the digital assistant or master bot itself per a built-in system intent or is to be handled as a different state in a current conversation flow.

(52) In certain embodiments, as part of this processing, the digital assistant determines if the user input utterance explicitly identifies a skill bot using its invocation name. If an invocation name is present in the user input, then it is treated as explicit invocation of the skill bot corresponding to the invocation name. In such a scenario, the digital assistant may route the user input to the explicitly invoked skill bot for further handling. If there is no specific or explicit invocation, in certain embodiments, the digital assistant evaluates the received user input utterance and computes confidence scores for the system intents and the skill bots associated with the digital assistant. The score computed for a skill bot or system intent represents how likely the user input is representative of a task that the skill bot is configured to perform or is representative of a system intent. Any system intent or skill bot with an associated computed confidence score exceeding a threshold value (e.g., a Confidence Threshold routing parameter) is selected as a candidate for further evaluation. The digital assistant then selects, from the identified candidates, a particular system intent or a skill bot for further handling of the user input utterance. In certain embodiments, after one or more skill bots are identified as candidates, the intents associated with those candidate skills are evaluated (according to the intent model for each skill) and confidence scores are determined for each intent. In general, any intent that has a confidence score exceeding a threshold value (e.g., 70%) is treated as a candidate intent. If a particular skill bot is selected, then the user utterance is routed to that skill bot for further processing. If a system intent is selected, then one or more actions are performed by the master bot itself according to the selected system intent.

(53) FIG. **2** is a simplified block diagram of a master bot (MB) system **200** according to certain embodiments. MB system **200** can be implemented in software only, hardware only, or a combination of hardware and software. MB system **200** includes a pre-processing subsystem **210**, a multiple intent subsystem (MIS) **220**, an explicit invocation subsystem (EIS) **230**, a skill bot invoker **240**, and a data store **250**. MB system **200** depicted in FIG. **2** is merely an example of an arrangement of components in a master bot. One of ordinary skill in the art would recognize many possible variations, alternatives, and modifications. For example, in some implementations, MB system **200** may have more or fewer systems or components than those shown in FIG. **2**, may combine two or more subsystems, or may have a different configuration or arrangement of subsystems.

(54) Pre-processing subsystem **210** receives an utterance "A" **202** from a user and processes the utterance through a language detector **212** and a language parser **214**. As indicated above, an utterance can be provided in various ways including audio or text. The utterance **202** can be a sentence fragment, a complete sentence, multiple sentences, and the like. Utterance **202** can include punctuation. For example, if the utterance **202** is provided as audio, the pre-processing subsystem **210** may convert the audio to text using a speech-to-text converter (not shown) that inserts punctuation marks into the resulting text, e.g., commas, semicolons, periods, etc.
(55) Language detector **212** detects the language of the utterance **202** based on the text of the utterance **202**. The manner in which the utterance **202** is handled depends on the language since each language has its own grammar and semantics. Differences between languages are taken into consideration when analyzing the syntax and structure of an utterance.
(56) Language parser **214** parses the utterance **202** to extract part of speech (POS) tags for individual linguistic units (e.g., words) in the utterance **202**. POS tags include, for example, noun (NN), pronoun (PN), verb (VB), and the like. Language parser **214** may also tokenize the linguistic units of the utterance **202** (e.g., to convert each word into a separate token) and lemmatize words. A lemma is the main form of a set of words as represented in a dictionary (e.g., "run" is the lemma for run, runs, ran, running, etc.). Other types of pre-processing that the language parser **214** can perform include chunking of compound expressions, e.g., combining "credit" and "card" into a single expression "credit card." Language parser **214** may also identify relationships between the words in the utterance **202**. For example, in some embodiments, the language parser **214** generates a dependency tree that indicates which part of the utterance (e.g., a particular noun) is a direct object, which part of the utterance is a preposition, and so on. The results of the processing performed by the language parser **214** form extracted information **205** and are provided as input to MIS **220** together with the utterance **202** itself.
(57) As indicated above, the utterance **202** can include more than one sentence. For purposes of detecting multiple intents and explicit invocation, the utterance **202** can be treated as a single unit even if it includes multiple sentences. However, in certain embodiments, pre-processing can be performed, e.g., by the pre-processing subsystem **210**, to identify a single sentence among multiple sentences for multiple intents analysis and explicit invocation analysis. In general, the results produced by MIS **220** and EIS **230** are substantially the same regardless of whether the utterance **202** is processed at the level of an individual sentence or as a single unit comprising multiple sentences.
(58) MIS **220** determines whether the utterance **202** represents multiple intents. Although MIS **220** can detect the presence of multiple intents in the utterance **202**, the processing performed by MIS **220** does not involve determining whether the intents of the utterance **202** match to any intents that have been configured for a bot. Instead, processing to determine whether an intent of the utterance **202** matches a bot intent can be performed by an intent classifier **242** of the MB system **200** or by an intent classifier of a skill bot (e.g., as shown in FIG. **3**). The processing performed by MIS **220** assumes that there exists a bot (e.g., a particular skill bot or the master bot itself) that can handle the utterance **202**. Therefore, the processing performed by MIS **220** does not require knowledge of what bots are in the chatbot system (e.g., the identities of skill bots registered with the master bot) or knowledge of what intents have been configured for a particular bot.
(59) To determine that the utterance **202** includes multiple intents, the MIS **220** applies one or more rules from a set of rules **252** in the data store **250**. The rules applied to the utterance **202** depend on the language of the utterance **202** and may include sentence patterns that indicate the presence of multiple intents. For example, a sentence pattern may include a coordinating conjunction that joins two parts (e.g., conjuncts) of a sentence, where both parts correspond to a separate intent. If the utterance **202** matches the sentence pattern, it can be inferred that the utterance **202** represents multiple intents. It should be noted that an utterance with multiple intents does not necessarily have different intents (e.g., intents directed to different bots or to different intents within the same bot).

Instead, the utterance could have separate instances of the same intent (e.g. “Place a pizza order using payment account X, then place a pizza order using payment account Y”).
(60) As part of determining that the utterance **202** represents multiple intents, the MIS **220** also determines what portions of the utterance **202** are associated with each intent. MIS **220** constructs, for each intent represented in an utterance containing multiple intents, a new utterance for separate processing in place of the original utterance, e.g., an utterance “B” **206** and an utterance “C” **208**, as depicted in FIG. **2**. Thus, the original utterance **202** can be split into two or more separate utterances that are handled one at a time. MIS **220** determines, using the extracted information **205** and/or from analysis of the utterance **202** itself, which of the two or more utterances should be handled first. For example, MIS **220** may determine that the utterance **202** contains a marker word indicating that a particular intent should be handled first. The newly formed utterance corresponding to this particular intent (e.g., one of utterance **206** or utterance **208**) will be the first to be sent for further processing by EIS **230**. After a conversation triggered by the first utterance has ended (or has been temporarily suspended), the next highest priority utterance (e.g., the other one of utterance **206** or utterance **208**) can then be sent to the EIS **230** for processing.
(61) EIS **230** determines whether the utterance that it receives (e.g., utterance **206** or utterance **208**) contains an invocation name of a skill bot. In certain embodiments, each skill bot in a chatbot system is assigned a unique invocation name that distinguishes the skill bot from other skill bots in the chatbot system. A list of invocation names can be maintained as part of skill bot information **254** in data store **250**. An utterance is deemed to be an explicit invocation when the utterance contains a word match to an invocation name. If a bot is not explicitly invoked, then the utterance received by the EIS **230** is deemed a non-explicitly invoking utterance **234** and is input to an intent classifier (e.g., intent classifier **242**) of the master bot to determine which bot to use for handling the utterance. In some instances, the intent classifier **242** will determine that the master bot should handle a non-explicitly invoking utterance. In other instances, the intent classifier **242** will determine a skill bot to route the utterance to for handling.
(62) The explicit invocation functionality provided by the EIS **230** has several advantages. It can reduce the amount of processing that the master bot has to perform. For example, when there is an explicit invocation, the master bot may not have to do any intent classification analysis (e.g., using the intent classifier **242**), or may have to do reduced intent classification analysis for selecting a skill bot. Thus, explicit invocation analysis may enable selection of a particular skill bot without resorting to intent classification analysis.
(63) Also, there may be situations where there is an overlap in functionalities between multiple skill bots. This may happen, for example, if the intents handled by the two skill bots overlap or are very close to each other. In such a situation, it may be difficult for the master bot to identify which of the multiple skill bots to select based upon intent classification analysis alone. In such scenarios, the explicit invocation disambiguates the particular skill bot to be used.
(64) In addition to determining that an utterance is an explicit invocation, the EIS **230** is responsible for determining whether any portion of the utterance should be used as input to the skill bot being explicitly invoked. In particular, EIS **230** can determine whether part of the utterance is not associated with the invocation. The EIS **230** can perform this determination through analysis of the utterance and/or analysis of the extracted information **205**. EIS **230** can send the part of the utterance not associated with the invocation to the invoked skill bot in lieu of sending the entire utterance that was received by the EIS **230**. In some instances, the input to the invoked skill bot is formed simply by removing any portion of the utterance associated with the invocation. For example, “I want to order pizza using Pizza Bot” can be shortened to “I want to order pizza” since “using Pizza Bot” is relevant to the invocation of the pizza bot, but irrelevant to any processing to be performed by the pizza bot. In some instances, EIS **230** may reformat the part to be sent to the invoked bot, e.g., to form a complete sentence. Thus, the EIS **230** determines not only that there is an explicit invocation, but also what to send to the skill bot when there is an explicit invocation. In

some instances, there may not be any text to input to the bot being invoked. For example, if the utterance was “Pizza Bot”, then the EIS **230** could determine that the pizza bot is being invoked, but there is no text to be processed by the pizza bot. In such scenarios, the EIS **230** may indicate to the skill bot invoker **240** that there is nothing to send.

(65) Skill bot invoker **240** invokes a skill bot in various ways. For instance, skill bot invoker **240** can invoke a bot in response to receiving an indication **235** that a particular skill bot has been selected as a result of an explicit invocation. The indication **235** can be sent by the EIS **230** together with the input for the explicitly invoked skill bot. In this scenario, the skill bot invoker **240** will turn control of the conversation over to the explicitly invoked skill bot. The explicitly invoked skill bot will determine an appropriate response to the input from the EIS **230** by treating the input as a stand-alone utterance. For example, the response could be to perform a specific action or to start a new conversation in a particular state, where the initial state of the new conversation depends on the input sent from the EIS **230**.

(66) Another way in which skill bot invoker **240** can invoke a skill bot is through implicit invocation using the intent classifier **242**. The intent classifier **242** can be trained, using machine-learning and/or rules-based training techniques, to determine a likelihood that an utterance is representative of a task that a particular skill bot is configured to perform. The intent classifier **242** is trained on different classes, one class for each skill bot. For instance, whenever a new skill bot is registered with the master bot, a list of example utterances associated with the new skill bot can be used to train the intent classifier **242** to determine a likelihood that a particular utterance is representative of a task that the new skill bot can perform. The parameters produced as result of this training (e.g., a set of values for parameters of a machine-learning model) can be stored as part of skill bot information **254**.

(67) In certain embodiments, the intent classifier **242** is implemented using a machine-learning model, as described in further detail herein. Training of the machine-learning model may involve inputting at least a subset of utterances from the example utterances associated with various skill bots to generate, as an output of the machine-learning model, inferences as to which bot is the correct bot for handling any particular training utterance. For each training utterance, an indication of the correct bot to use for the training utterance may be provided as ground truth information. The behavior of the machine-learning model can then be adapted (e.g., through back-propagation) to minimize the difference between the generated inferences and the ground truth information.

(68) In certain embodiments, the intent classifier **242** determines, for each skill bot registered with the master bot, a confidence score indicating a likelihood that the skill bot can handle an utterance (e.g., the non-explicitly invoking utterance **234** received from EIS **230**). The intent classifier **242** may also determine a confidence score for each system level intent (e.g., help, exit) that has been configured. If a particular confidence score meets one or more conditions, then the skill bot invoker **240** will invoke the bot associated with the particular confidence score. For example, a threshold confidence score value may need to be met. Thus, an output **245** of the intent classifier **242** is either an identification of a system intent or an identification of a particular skill bot. In some embodiments, in addition to meeting a threshold confidence score value, the confidence score must exceed the next highest confidence score by a certain win margin. Imposing such a condition would enable routing to a particular skill bot when the confidence scores of multiple skill bots each exceed the threshold confidence score value.

(69) After identifying a bot based on evaluation of confidence scores, the skill bot invoker **240** hands over processing to the identified bot. In the case of a system intent, the identified bot is the master bot. Otherwise, the identified bot is a skill bot. Further, the skill bot invoker **240** will determine what to provide as input **247** for the identified bot. As indicated above, in the case of an explicit invocation, the input **247** can be based on a part of an utterance that is not associated with the invocation, or the input **247** can be nothing (e.g., an empty string). In the case of an implicit invocation, the input **247** can be the entire utterance.

(70) Data store **250** comprises one or more computing devices that store data used by the various subsystems of the master bot system **200**. As explained above, the data store **250** includes rules **252** and skill bot information **254**. The rules **252** include, for example, rules for determining, by MIS **220**, when an utterance represents multiple intents and how to split an utterance that represents multiple intents. The rules **252** further include rules for determining, by EIS **230**, which parts of an utterance that explicitly invokes a skill bot to send to the skill bot. The skill bot information **254** includes invocation names of skill bots in the chatbot system, e.g., a list of the invocation names of all skill bots registered with a particular master bot. The skill bot information **254** can also include information used by intent classifier **242** to determine a confidence score for each skill bot in the chatbot system, e.g., parameters of a machine-learning model.
(71) FIG. **3** is a simplified block diagram of a skill bot system **300** according to certain embodiments. Skill bot system **300** is a computing system that can be implemented in software only, hardware only, or a combination of hardware and software. In certain embodiments such as the embodiment depicted in FIG. **1**, skill bot system **300** can be used to implement one or more skill bots within a digital assistant.
(72) Skill bot system **300** includes an MIS **310**, an intent classifier **320**, and a conversation manager **330**. The MIS **310** is analogous to the MIS **220** in FIG. **2** and provides similar functionality, including being operable to determine, using rules **352** in a data store **350**: (1) whether an utterance represents multiple intents and, if so, (2) how to split the utterance into a separate utterance for each intent of the multiple intents. In certain embodiments, the rules applied by MIS **310** for detecting multiple intents and for splitting an utterance are the same as those applied by MIS **220**. The MIS **310** receives an utterance **302** and extracted information **304**. The extracted information **304** is analogous to the extracted information **205** in FIG. **1** and can be generated using the language parser **214** or a language parser local to the skill bot system **300**.
(73) Intent classifier **320** can be trained in a similar manner to the intent classifier **242** discussed above in connection with the embodiment of FIG. **2** and as described in further detail herein. For instance, in certain embodiments, the intent classifier **320** is implemented using a machine-learning model. The machine-learning model of the intent classifier **320** is trained for a particular skill bot, using at least a subset of example utterances associated with that particular skill bot as training utterances. The ground truth for each training utterance would be the particular bot intent associated with the training utterance.
(74) The utterance **302** can be received directly from the user or supplied through a master bot. When the utterance **302** is supplied through a master bot, e.g., as a result of processing through MIS **220** and EIS **230** in the embodiment depicted in FIG. **2**, the MIS **310** can be bypassed so as to avoid repeating processing already performed by MIS **220**. However, if the utterance **302** is received directly from the user, e.g., during a conversation that occurs after routing to a skill bot, then MIS **310** can process the utterance **302** to determine whether the utterance **302** represents multiple intents. If so, then MIS **310** applies one or more rules to split the utterance **302** into a separate utterance for each intent, e.g., an utterance "D" **306** and an utterance "E" **308**. If utterance **302** does not represent multiple intents, then MIS **310** forwards the utterance **302** to intent classifier **320** for intent classification and without splitting the utterance **302**.
(75) Intent classifier **320** is configured to match a received utterance (e.g., utterance **306** or **308**) to an intent associated with skill bot system **300**. As explained above, a skill bot can be configured with one or more intents, each intent including at least one example utterance that is associated with the intent and used for training a classifier. In the embodiment of FIG. **2**, the intent classifier **242** of the master bot system **200** is trained to determine confidence scores for individual skill bots and confidence scores for system intents. Similarly, intent classifier **320** can be trained to determine a confidence score for each intent associated with the skill bot system **300**. Whereas the classification performed by intent classifier **242** is at the bot level, the classification performed by intent classifier **320** is at the intent level and therefore finer grained. The intent classifier **320** has

access to intents information **354**. The intents information **354** includes, for each intent associated with the skill bot system **300**, a list of utterances that are representative of and illustrate the meaning of the intent and are typically associated with a task performable by that intent. The intents information **354** can further include parameters produced as a result of training on this list of utterances.

(76) Conversation manager **330** receives, as an output of intent classifier **320**, an indication **322** of a particular intent, identified by the intent classifier **320**, as best matching the utterance that was input to the intent classifier **320**. In some instances, the intent classifier **320** is unable to determine any match. For example, the confidence scores computed by the intent classifier **320** could fall below a threshold confidence score value if the utterance is directed to a system intent or an intent of a different skill bot. When this occurs, the skill bot system **300** may refer the utterance to the master bot for handling, e.g., to route to a different skill bot. However, if the intent classifier **320** is successful in identifying an intent within the skill bot, then the conversation manager **330** will initiate a conversation with the user.

(77) The conversation initiated by the conversation manager **330** is a conversation specific to the intent identified by the intent classifier **320**. For instance, the conversation manager **330** may be implemented using a state machine configured to execute a dialog flow for the identified intent. The state machine can include a default starting state (e.g., for when the intent is invoked without any additional input) and one or more additional states, where each state has associated with it actions to be performed by the skill bot (e.g., executing a purchase transaction) and/or dialog (e.g., questions, responses) to be presented to the user. Thus, the conversation manager **330** can determine an action/dialog **335** upon receiving the indication **322** identifying the intent and can determine additional actions or dialog in response to subsequent utterances received during the conversation.

(78) Data store **350** comprises one or more computing devices that store data used by the various subsystems of the skill bot system **300**. As depicted in FIG. **3**, the data store **350** includes the rules **352** and the intents information **354**. In certain embodiments, data store **350** can be integrated into a data store of a master bot or digital assistant, e.g., the data store **250** in FIG. **2**.

(79) Scalable Search and Content-Based Schema Linking

(80) As discussed above, in order to perform well, a natural language to logical form translator including an NL2SQL model should be able to accurately correlate words or tokens in the user's natural language query with entities, attributes, and/or values in the database the user is querying. Conventional NL2SQL translators generally rely on schema linking in which values extracted from the user's natural language query are matched to components of the database's schema (e.g., tables, columns, values, etc.). However, in many cases, an exact match is not found in the database's schema (i.e., a value extracted from the user's query may correspond to a synonym of or may be similar to a value in the database schema). As a result, these NL2SQL translators sometimes fail to provide satisfactory responses to the user's natural language query. Conventional approaches have attempted to solve this problem by relying on a SQL similarity operator such as the SQL LIKE operator, but this approach has often led to reduced NL2SQL translator performance and slow responses to the user's natural language query.

(81) To overcome these challenges and others, techniques are described herein for preprocessing data assets to be used in a natural language to logical form model (e.g., an NL2SQL model) based on scalable search and content-based schema linking. More specifically, to achieve a practical text-to-logical form workflow, a model needs to correlate natural language queries with a given database. Therefore, schema linking is considered helpful for text-to-logical form parsing. As used herein, schema linking means identifying references of columns, tables, and values, from the database, in natural language utterances. For example, for the utterance “Tell me all the names of people that have paid their dues with an amount equal to or above two-hundred and fifty dollars”, “names” is a column reference to people_name within the database, “dues” is a table reference to

homeowners_association within the database, and “two-hundred and fifty dollars” is a value reference to value within the database. As described herein in further detail, scalable searches are performed on value lists for a given database using content of the utterance (e.g., words or tokens from the utterance) to identify references of columns and tables from the database, and values from the database, in the utterance. The scalable search is executed by a search engine on an index (e.g., an inverted index), which allows for the scalability and searching on large sets of data or databases. The results of the scalable searches are then combined with named entities identified within the utterance in a composite data structure. The composite data structure is appended to the utterance and provided as input to a natural language to logical form model in order to make the natural language to logical form model explicitly aware of what tables, columns, and values are involved in the utterance. The named entities identified in the composite data structure correspond with columns, tables, and/or values for the database in the utterance and provide an additional layer of schema linking beyond the scalable search. The schema linking helps both cross-domain generalizability and complex logical form generation, which are present issues of text-to-logical form workflows.

(82) FIG. **4**A shows a scalable search and content-based schema linking architecture **400** for preprocessing an utterance **405** in accordance with various embodiments. The scalable search and content-based schema linking architecture **400** comprises a skill **410**, a dataset **415** including a database schema and one or more value lists, a named entity recognition system **420**, a scalable search and value list linker **425**, and a data structure generator **430**. The utterance **405** is obtained from one or more sources such as a database (not shown), a computing system (e.g., data preprocessing subsystem), a user, or the like. In some instances, the utterance **405** is obtained from a user such as a user interacting with the digital assistant, as described herein with respect to FIGS. **1-3**. In some instances, the utterance **405** corresponds to a natural language statement, query and/or question such as “Is Peter allergic to nuts?” or “thirty-five cents and twenty cats of Peter has been here today noon”. The skill **410** is a bot (e.g., the chatbot as described with respect to FIGS. **1-3**) or any other artificial intelligence-based computer program that can respond to natural language utterances (e.g., questions or comments). The skill **410** may be configured to allow end users to communicate with a system using natural language utterances (e.g., execute a natural language query on a database).

(83) The data set **415** includes information describing one or more databases (e.g., a relational database). Relational databases range considerably in their complexity and size, but typically are comprised of one or more themed tables (also known as relations) with schematic uniqueness and relatedness through the assignment of primary keys and foreign keys, respectively. Within the tables there are rows (also known as tuples) and columns (also known as attributes). Tables are the functional units within a database schema. Each row from a table is unique and represents a unit of relevance. Columns serve multiple functions within a table, including the enforcement of each row's uniqueness, linkage to other tables, indexing, and information about each row or its raw values/measurements. The structure of the database according to these various entities (table, row, column, values, etc.) may be stored in a data structure such as a database schema.

(84) A database schema is a collection of logical data structures or schema objects that directly refer to the various entities in the database. Schema objects are logical structures created by users or automated systems (e.g., a database management system). Objects such as tables or indexes hold data, or can be comprised of a definition only, such as a view. An index is a data structure that a database management system uses to improve query performance by speeding up data retrieval. Indexes are built by a user or the database management system on one or more columns of a table and each index maintains a list of values (value list) within the one or more columns that may be sorted in ascending or descending order. In some instances, the index also stores synonyms or alternative values for a given value. The synonyms can be defined by a user or automatically obtained by the database management system from one or more sources such as a dictionary or

thesaurus. In some instances, the index also stores pointers which are reference information for the location of additional information concerning the table and rows associated with the one or more columns and values thereof. For example, a pointer may be mapped to a row in a given table to allow the database management system to find the specific row and retrieve values associated with other columns or attributes from the specific row. With reference to FIG. **4**A, an exemplary data set **415** may include an index generated for a column or attribute “Allergy”. The index stores a list of values for the attribute including Cat, Dog, Egg, Nuts, and Fish, and any synonyms for the values such as Cats, Feline, Dogs, Canine, Tree Nuts, Nut, Flounder, Bass, etc. The data set **415** may be obtained from one or more sources such as a user (e.g., a customer), a database, a database management system, a separate storage device, or the like.

(85) The named entity recognition system **420** performs named entity recognition on the utterance **405** to identify entities in the utterance **405** and classify them into predefined categories or classes. Entities are words in a text such as utterance **405** that correspond to a specific type of data. The entities can be numerical, such as cardinal numbers; temporal, such as dates; nominal, such as names of people and places; and political, such as geopolitical entities. Basically, an entity can be anything the user or designer wishes to designate as an item in a text that has a corresponding label. In some instances, the categories or classes of the entities are predefined to correspond with table names or attribute names within the database schema. A few common entities used as examples herein are domain independent, called System Entities: Organizations or Companies Quantities, Percentages, or Numbers Currency People Geographic Locations Dates and Times

(86) Named entity recognition is the process by which the named entity recognition system **420** takes an input of unstructured data (an utterance) and outputs structured data, specifically the identification of named entities. For example, for an utterance “Was Peter a manager in the product development department of the ABC Company in 2018?,” there are several potential entities. First, there is the nominal entity: “Peter”. A corresponding class label for such an entity may be PERSON. There is another nominal entity: “ABC Company”. A corresponding class labels for such an entity may be ORGANIZATION. Finally, there is a temporal entity, “2018”. A corresponding class labels for such an entity may be DATE. In some instances, the identification of the identities includes the extraction of matched offsets (described herein as Begin Offset and End Offset) for the named entities, which are the numerical beginning and ending position of the named entities within the utterance. There are typically three approaches for named entity recognition—dictionary based, rule based, and machine learning based. A dictionary-based approach uses large databases of named-entities and possibly trigger terms of different categories as a reference to locate and tag entities in a given text. A rule-based approach uses handcrafted rules to capture named-entities and classify them based on their orthographic and morphological features. A machine learning based approach identifies and classifies named entities in the text utilizing supervised, semi-supervised, and unsupervised learning techniques. The supervised techniques may use Support Vector Machines, Hidden Markov models, Decision trees, Naive Bayesian methods, Conditional Random Fields (CRF), and various deep learning models such as feed-forward neural networks (FFNN), recurrent neural networks (RNN), convolution neural networks (CNN), bidirectional encoder representations from transformers (BERT), Long Short-Term Memory (LSTM) neural networks, or combinations thereof such as bidirectional long short-term memory networks (Bi-LS™) and CRFs models. In certain instances, the named entity recognition system **420** comprises one or more Bi-LS™ neural networks, one or more CNNs, one or more CRFs, and/or one or more BERTs trained for named entity recognition. The foregoing named entity recognition approaches are not intended to be limiting and the named entity recognition system **420** may implement any approach or combination of approaches for identifying and classifying named entities in an utterance.

(87) Entities and classes thereof identified from named entity recognition approaches (e.g., one or more machine learning models) are used by the named entity recognition system **420** to identify table name or attribute name matches within the database schema (described herein as a CBSL

match) and extract out information for each of the entities from the database schema. In other words, the classes for the entities may be predefined to match table names and attribute names within the database schema, and thus classifying the named entities within the utterance essentially performs a layer of schema linking (identifying references of columns, tables and values in an utterance). For example, named entity recognition approaches within the named entity recognition system **420** may be used to identify System Entities with the predefined classes: TIME, DATE, NUMBER, CURRENCY, and/or PERSON, and extract information from the database schema for each of the System Entities. The information extracted may include a predefined format for values under the System Entities: TIME “date”: date in milliseconds “hrs”: hour value “mins”: minute value “secs”: seconds value “hourFormat”: AM or PM “zone”: time zone “zoneOffset”: time zone offset DATE “date”: date in current milliseconds NUMBER “number”: value converted to number CURRENCY “amount”: currency amount in number “currency”: currency name “totalCurrency”: value of the currency as a string PERSON “name”: name of a person

(88) The scalable search and value list linker **425** performs a scalable search for words or tokens from the utterance **405** on the value lists within the data set **415** to determine whether any words or tokens from the utterance **405** correspond to attributes or values of a table. As used herein, a scalable search refers to a search on an index for a word or token in an utterance that is identical to a word or token in a group of predefined values and/or related to and/or derived from the values in the group of predefined values (e.g., a value and its family/synonyms). The scalable search can be performed using one or more search engines configured to perform word or token matching between sets of words or tokens. In some instances, the one or more search engines can perform token matching between sets of words or tokens using one or more exact string-matching algorithms and one or more approximate string-matching algorithms. An example of a search engine that is configured to perform such token matching is Elasticsearch. Elasticsearch is a distributed, free and open search and analytics engine for all types of data, including textual, numerical, geospatial, structured, and unstructured. Additional information for Elasticsearch can be found in “Elasticsearch: The Definitive Guide: A Distributed Real-time Search and Analytics Engine” by Gormley et al., published by O'Reilly Media, the entire contents of which are hereby incorporated by reference as if fully set forth herein. The scalable search may be set up by ingesting data (e.g., the value lists) from a user or the database schema into the search engine and storing and indexing the ingested data according to mappings (which may be user-specified, and/or automatically derived from the database schema) to make the ingested data searchable.

(89) Searches may be performed on the indexed data by inputting the utterance **405** into the search engine, preprocessing the utterance including tokenizing, normalizing, stemming, stop word removal, or combinations thereof, and executing queries on the indexed data using the preprocessed utterances (e.g., tokens). In some instances, the preprocessing includes tokenizing, using a tokenizer, the utterance **405** into a set of tokens. The tokenizer can be configured to generate the set of tokens such that the tokens are split by white space, special characters, a sequence of words or numbers, or any combination thereof. The scalable search and value list linker **425** will execute queries on the indexed data using the preprocessed utterances (e.g., tokens), and output value matches comprising: (i) any value that matches a token or word from the utterance and any attribute associated with the value (described herein as Canonical Value or Name; or Entity Value or Name), and (ii) the matched offsets (described herein as Begin Offset and End Offset) for the matching token or word (described herein as Original String), which are the numerical beginning and ending position of the word or token within the utterance. The output from the scalable search and value list linker **425** comprises one or more value matches (described herein as CBSL matches). In some instances, only values that are within the value lists will be matched, as an exact match on words or tokens from the utterance. The value matches may be case insensitive, ignore consecutive white spaces, consider numbers/characters as separate tokens, and/or other similar type parameters. For example: Case Insensitive: “hamburger” matches with “Hamburger”

Ignore consecutive white spaces: “Hamburger restaurant” matches with “Hamburger Restaurant” Does not ignore special characters: “Hamburger.restaurant” does not match with “Hamburger Restaurant” Numbers/Characters as separate tokens: “401K” matches with “401” and “401K” and “K” Numbers/Characters as separate tokens: “AAA123” matches with “AAA”, “123”, “AAA123”, but does not match with “A1”, “AA”, “12” Exact match on tokens: “Motor sport” does not match with “Motorsport” because the “motor”, “sport” tokens are not equal to “Motorsport” token Exact match on tokens: “Sept” does not match with “September”

(90) Any given synonyms from within the value list will be matched with tokens from the utterance, and the values will be returned by the scalable search and value list linker **425** as the Canonical Value or Name; or Entity Value or Name. For example, for a value list that includes the example entry: Value “Motorsports”; Synonym “MotoX”; and a received an Utterance: “How many employees are in MotoX?”, the following will be matched:

(91) TABLE-US-00001 { “utterance”: “How many employees are in MotoX?” “CBSLMatches”: { “corporations”: [{ “canonicalName”: “motorsports”, “originalString”: “MotoX”,

(92) where the Synonym: “MotoX” from within the value list will be matched with token: “MotoX” from the utterance, and the value returned is the Canonical Value or Name: “Motorsports”. According to an embodiment, in the above instance, for the utterance: “How many employees are in motor sports?”, no matches will be returned because “motor sports” is not provided as a synonym of “Motorsports”. According to another embodiment, a fuzzy matching technique may be implemented to match “motor sports” with “Motorsports” given the similarities between the terms when white space is not considered. As another example, for the utterance: “How many employees ride motorcycles?”, no matches will be returned using either exact matching or fuzzy matching because “motorcycles” is not provided as a synonym of “Motorsports” and “motorcycles” is not similar in structure to any of the terms in the value list.

(93) Synonyms can be any value and can be duplicate to other Canonical Values or Names (e.g., (<Value>: <Synonym>, <Synonym2> . . .)). For example, for attribute=FastFood: (“Restaurant ABC”: “chicken”), (“Restaurant DEF”: “chicken”), (. . . the synonym “chicken” is a duplicate for Canonical Values or Names “Restaurant ABC” and “Restaurant DEF”. In such a case, all matches will be returned by the scalable search and value list linker **425** as follows:

(94) TABLE-US-00002 { “utterance”: “chicken”, “CBSLMatches”: { “FastFood”: [{ “canonicalName”: “Restaurant ABC”, “originalString”: “chicken”, “beginOffset”: 0, “endOffset”: 7 }, { “canonicalName”: “Restaurant DEF”, “originalString”: “chicken”, “beginOffset”: 0, “endOffset”: 7 }] } }.

(95) In some instances, if a match is a sub token of another value in the same entity, the scalable search and value list linker **425** will return only the longest match (e.g., (<Value>: <Synonym>, <Synonym2> . . .)). For example, for attribute=Company: (“Alpha”), (“Restaurant Alpha”), (. . . where “Alpha” is a value in the entity, only “Restaurant Alpha” will be returned because it's the longest match as follows:

(96) TABLE-US-00003 { “utterance”: “Restaurant Alpha”, “CBSLMatches”: { “Company”: [{ “canonicalName”: “Restaurant Alpha”, “originalString”: “Restaurant Alpha”, “beginOffset”: 0, “endOffset”: 9 }] } }.

(97) In some instances, when a match of one entity is a sub token of another entity, the scalable search and value list linker **425** will return both entity matches (e.g., (<Value>: <Synonym>, <Synonym2> . . .)). For example:

(98) TABLE-US-00004 Attribute = Country: (“USA”: “United States of America”, “America”), (. . . Attribute = Flight: (“AF”: “America Flyers”, “America”), (. . . will return: { “utterance”: “america flyers”, “CBSLMatches”: { “Flight”: [{

“canonicalName”: “AF”, “originalString”: “america flyers”, “beginOffset”: 0, “endOffset”: 16 }], “Country”: [{ “canonicalName”: “USA”, “originalString”: “america”, “beginOffset”: 0, “endOffset”: 7 }] } }.

(99) because “America” is a subset of “America Flyers”, but in this instance Country and Flight are different entities and both matches will be returned.

(100) The data structure generator **430** organizes and stores the CBSL match results obtained by the named entity recognition system **420** and the CBSL match results obtained by the scalable search and value list linker **425** in a composite data structure. The data structure generator **430** generates the data structure in a predefined format. In some instances, the predefined format includes: the utterance in a first position; a list of each named entity recognized in the respective utterance, a position indicator that indicates where in the respective utterance each named entity can be found, and the word or token for the respective named entity in one or more second positions; and a list of each value identified for a respective word or token in the utterance, a position indicator that indicates where in the respective utterance each word or token term can be found, and the word or token from the utterance in one or more third positions. The following is an example of a data structure generated in this predefined format by the data structure generator **430** for the utterance: “Thirty-five cents and twenty cats of Peter has been here today noon”:

(101) TABLE-US-00005 { “utterance”: “thirty-five cents and twenty cats of Peter has been here today noon”, “CBSLMatches”: { “CURRENCY”: [{ “beginOffset”: 0, “endOffset”: 17, “originalString”: “thirty-five cents”, “amount”: 0.35, “currency”: “dollar”, “totalCurrency”: “0.35 dollar” }], “DATE”: [{ “beginOffset”: 57, “endOffset”: 67, “originalString”: “today noon”, “date”: 1636416000000 }], “NUMBER”: [{ “beginOffset”: 22, “endOffset”: 28, “originalString”: “twenty”, “number”: 20 }], “PERSON”: [{ “beginOffset”: 37, “endOffset”: 42, “originalString”: “Peter”, “name”: “peter” }], “TIME”: [{ “beginOffset”: 57, “endOffset”: 67, “originalString”: “today noon”, “date”: 1636459200000, “hrs”: 12, “mins”: 0, “secs”: 0, “hourFormat”: “PM”, “zone”: “UTC”, “zoneOffset”: “0” }], “Allergy”: [{ “canonicalName”: “cat”, “originalString”: “cats”, “beginOffset”: 29, “endOffset”: 33 }] } }.

(102) For this exemplary data structure, the named entity recognition system **420** identified System Entities: “thirty-five cents”, “today noon”, “twenty”, and “Peter”, extracted the offsets for the System Entities within the utterance, and classified the System Entities as CURRENCY, DATE/TIME, NUMBER, and PERSON, respectively. The named entity recognition system **420** confirmed the System Entities as attributes within the database schema based on CBSL matching and extracted a defined format for values under the attributes, e.g., “number” being a number value “20” as compared to the “originalString” value of “twenty” and “currency” being a dollar value “0.5 dollar” as compared to the “originalString” value of ““thirty-five cents”. The data structure generator **430** obtained all this information from the named entity recognition system **420** and populated the information into the data structure in a predefined format. The scalable search and value list linker **425** identified token: “cats” and extracted the offsets for the token within the utterance. The scalable search and value list linker **425** confirmed the token as a value within the database schema based on CBSL matching, extracted a value (“canonicalName”: “cat”) corresponding to the token, and extracted database schema information (e.g., attribute name=“Allergy”) associated with the value. The data structure generator **430** obtained all this information from the scalable search and value list linker **425** and populated the information into the data structure in the predefined format to generate the composite data structure.

(103) The predefined format for the data structure includes keys and values. The keys are populated based on attribute or Entity Names: e.g., entity_name, in the database schema. The values are populated based on the predefined format for a value and/or entity names: e.g., predefined format for “date” or Canonical Name, in the database schema.
(104) For example, in the following database schema, one of the attributes is “entity_name”: “parties.party_name” and the predefined format is:
(105) TABLE-US-00006 { “name”: “parties”, “attributes”: [{ “name”: “party_id”, “type”: “number” }, { “name”: “party_name”, “type”: “entity”, “entity name”: “parties.party name” }, { “name”: “parties_party_sites_back_link″ “type”: “composite_entity”, “entity_name”: “party_sites”, “multiple values”: true }] }.
(106) For example, for an original string “Advanced Restaurant Devices” and Canonical Name “Advanced Restaurant Device”, a generated data structure would be:
(107) TABLE-US-00007 { “CBSLMatches”: { “parties.party name”: [{ “canonicalName”: “Advanced Restaurant Devices”, “originalString”: “Advanced Restaurant Devices”, “beginOffset”: 21, “endOffset”: 45 }] } }, where the key field for the attribute is populated with the Entity Name: “parties. party_name” and the value field is populated with the Canonical Name: “Advanced Restaurant Devices”.
(108) In the same example, if the “entity_name” is changed to “custom_name” in the database schema, then the generated data structure would be:
(109) TABLE-US-00008 { “CBSLMatches”: { ”custom name”: [{ “canonicalName”: “Advanced Restaurant Devices”, “originalString”: “Advanced Restaurant Devices”, “beginOffset”: 21, “endOffset”: 45 }] } }, where the key field for the attribute is populated with the Entity Name: “custom_name” and the value field is populated with the Canonical Name: “Advanced Restaurant Devices”.
(110) In addition, it is possible that the same Entity Name be used for multiple attributes in different tables. This is known as many-to-one matching. For example, for the following database schema:
(111) TABLE-US-00009 { “name”: “parties”, “attributes”: [{ “name”: “party_id”, “type”: “number” }, { “name”: “party_name” “type”: “entity”, “entity name”: “custom name” }, { “name”: “parties_party_sites_back_link”, “type”: “composite_entity”, “entity_name”: “party_sites”, “multiple_values”: true }] }, { “name”: “party_sites”, “attributes”: [{ “name”: “parties”, “type”: “composite_entity”, “entity_name”: “parties”, “multiple_values”: true }, { “name”: “party_site_id”, “type”: “number” }, { “name”: “party_site_name”, “type”: “entity”, “entity_name”: “custom_name” }, { “name”: “party_site_accounts_payable_invoices_back_link”, “type”: “composite_entity”, “entity _name”: “accounts_payable_invoices″, “multiple_values”: true }] }, where, in this case, both parties.party_name and party_sites.party_site_name have the same entity_name i.e., custom_name. In this instance, if the scalable search and value list linker **425** finds matches with both parties.party_name and party_sites.party_site_name, the scalable search and value list linker **425** returns the longest match, as shown in the following generated data structure:
(112) TABLE-US-00010 { “CBSLMatches”: { “custom name”: [{ “canonicalName”: “Advanced Restaurant Devices”, “originalString”: “Advanced Restaurant Devices”, “beginOffset”: 21, “endOffset”: 45

}] } }.

(113) In some instances, a value list can be used across different attribute usages, where the EntityAttribute-ValueList can be a many-to-one relation and the value list name does not have to match with attribute names within the database schema. As shown in FIG. **4**B, a user interface **450** may be provided (e.g., provided by a database management system or a digital assistant system) that allows for a user to interact and define with objects of a database schema. For this exemplary schema, an attribute has a canonicalName=Location, a PrimaryName=Location, but the ReferencedEntity (the ValueList name) is “state” which is a completely different name from the canonicalName and PrimaryName of the attribute. In some instances, the CBSL match retuned for the attribute or value is the name of the referenced usage (i.e., referenceUsage). For example, with respect to the database schema shown in FIG. **4**B, where the base schema contains the referenceUsage of “state” linked to Location, the CBSL match would return the name “state” for the attribute or value. As such, if a user provides an utterance “Who lives in California?” and the base schema is defined as:

(114) TABLE-US-00011 { “name” : “Location”, “type” : “entity”, “entity_name” : “state”, “multiple_values” : false, “fuzzy_match” : false }

(115) The data structure for the CBSL match would look like the following:

(116) TABLE-US-00012 “cbslMatches”: { “state”: [{ “endOffset”: 23, “originalString”: “california”, “canonicalName”: “california”, “beginOffset”: 13 }] } where “state” is provided as the attribute, and the expected logical form (e.g., Oracle Mark Up Language (OMRL)) would be expected to look like the following:

(117) TABLE-US-00013 “omrl”: { “omrql”: “SELECT name FROM Employee WHERE Location = ‘california’”, }.

(118) Once the data structure is generated, it can be appended to the utterance **405** for assisting with downstream processing of the utterance **405**. For example, the data structure may be used to facilitate encoding and decoding of the input utterance into a logical form by a natural language to logical form (NL-LF) translator **435**. More specifically, a NL-LF translator **435** can use the data structure to parse the utterance **405** and link the words or tokens of the utterance **405** to entities, attributes, and values of a database schema, which can then be used to encode and decode the input utterance into a logical form such as a SQL query. In some instances, the NL-LF translator **435** can include one or more NL-LF models such as one or more NL2SQL models. Examples of an NL2SQL model include, but are not limited to, RAT-SQL and DuoRAT. Additional information for the RAT-SQL model is found in “RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers” by Wang et al., published in Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, the entire contents of which are hereby incorporated by reference as if fully set forth herein. Additional information for the DuoRAT model is found in “DuoRAT: Towards Simpler Text-to-SQL Models” by Scholak et al., published in Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics, the entire contents of which are hereby incorporated by reference as if fully set forth herein. Thereafter, the logical form can be executed on a database associated with the database schema. In some instances, executing the logical form query includes searching one or more databases and returning one or more responses in response to the utterance **405**. In some instances, the one or more responses can be output to the user via a user interface of the digital assistant or a display. Advantageously, by using scalable search and CBSL matches as described herein, the NL-LF translator **435** response time and accuracy can be improved over conventional standardized programming language techniques such as SQL searches on the database using a SQL similarity operator.

(119) FIG. **5** illustrates an example process **500** for preprocessing utterances based on scalable search and content-based schema linking according to various embodiments. The processing

depicted in FIG. **5** may be implemented in software (e.g., code, instructions, a program) executed by one or more processing units (e.g., one or more processors, cores) of the respective systems, hardware, or combinations thereof described throughout. The software may be stored on a non-transitory storage medium (e.g., on a memory device). Although the methods presented in FIG. **5** depict the various processing steps occurring in a particular sequence or order, this is not intended to be limiting. In certain alternative embodiments, the steps may be performed in parallel and/or in a different order. In certain embodiments, such as in the embodiments depicted in FIGS. **1**-**4**B, the processing depicted in FIG. **5** may be performed by a pre-processing subsystem (e.g., pre-processing subsystem **210** and/or architecture **400**) to generate data structures for facilitating translation of natural language utterances to a logical form.

(120) At block **502**, an utterance is accessed. In some instances, the utterance corresponds to a natural language statement, query and/or question. In some instances, the utterance is obtained from one or more sources such as a database (not shown), a computing system (e.g., data preprocessing subsystem), a user, or the like. In some instances, the user is a user interacting with the digital assistant, as described herein with respect to FIGS. **1**-**3**.

(121) At block **504**, one or more named entities within the utterance are classified into predefined classes. In some instances, the classifying results in each of the one or more named entities being assigned a class from the predefined classes where the predefined classes match table names or attribute names within a database schema. In some instances, the classes for the entities may be predefined to match table names and attribute names within the database schema, and thus classifying the named entities within the utterance essentially performs a layer of schema linking (identifying references of columns, tables and values in an utterance). In some instances, the classifying extracts matched offsets for each of the one or more named entities, which are a numerical beginning and an ending position of the each of the one or more named entities within the utterance.

(122) At block **506**, information from the database schema is extracted for each of the one or more named entities based on the class assigned to each of the one or more named entities. In some instances, the information includes a predefined format for values within a table or attribute associated with the class via the matching table name or attribute name. In some instances, the information is extracted for system entities. In some instances, the system entities include TIME, DATE, NUMBER, CURRENCY, and/or PERSON and the information extracted may include a predefined format for values under the system entities.

(123) At block **508**, one or more value lists within the database schema are searched using tokens from the utterance. In some instances, the searching identifies and outputs value matches comprising: (i) any value within the one or more value lists that matches a token from the utterance and (ii) any attribute associated with a matching value. In some instances, the one or more value lists are structured within one or more indexes, and the search is performed by running queries on the one or more indexes using the tokens from the utterance. In some instances, the one or more value lists includes one or more attributes, values, and synonyms of the values, and the searching identifies and outputs value matches comprising: (i) any value within the one or more value lists that matches a token from the utterance, (ii) any value within the one or more value lists that has a synonym of the value that matches a token from the utterance, and (iii) any attribute associated with a matching value. In some instances, the searching extracts matched offsets for each of the tokens from the utterance, which are a numerical beginning and an ending position of the each of the tokens in the utterance. In some instances, the search can be performed using one or more search engines configured to perform word or token matching between sets of words or tokens using one or more exact string-matching algorithms and one or more approximate string-matching algorithms. In some instances, the search may be set up by ingesting data (e.g., the value lists) from a user or the database schema into the search engine and storing and indexing the ingested data according to user-specified mappings (which may be automatically derived from the database

schema) to make the ingested data searchable. In some instances, searches may be performed on the indexed data by inputting the utterance into the search engine, preprocessing the utterance including tokenizing, normalizing, stemming, stop word removal, or combinations thereof, and executing queries on the indexed data using the preprocessed utterances (e.g., tokens). In some instances, the preprocessing includes tokenizing, using a tokenizer, the utterance into a set of tokens. The tokenizer can be configured to generate the set of tokens such that the tokens are split by white space, special characters, a sequence of words or numbers, or any combination thereof.
(124) At block **510**, a data structure is generated. In some instances, the data structure is generated by organizing and storing: (i) each of the one or more named entities and the assigned class for each of the one or more named entities, (ii) each of the value matches and the token matching each of the value matches, and (iii) the utterance, in a predefined format for the data structure. In some instances, generating the data structure includes organizing and storing the matched offsets for each of the tokens in the predefined format for the data structure, organizing and storing the matched offsets for each of the one or more named entities in the predefined format for the data structure, and formatting each of the one or more named entities in the predefined format associated with the class assigned to each of the one or more named entities. In some instances, the predefined format includes: the utterance in a first position; a list of each named entity recognized in the respective utterance, a position indicator that indicates where in the respective utterance each named entity can be found, and the word or token for the respective named entity in one or more second positions; and a list of each value identified for a respective word or token in the utterance, a position indicator that indicates where in the respective utterance each word or token term can be found, and the word or token from the utterance in one or more third positions.
(125) At block **512**, the data structure is output.
(126) At block **514**, a database is searched based on the data structure. In some instances, in order to search the databases, the data structure is input into a machine learning model, the utterance is translated, using the machine learning model, into a logical form based on the data structure, the logical form is executed as a query on a database associated with the database schema to retrieve a result for the query, and the result of the utterance is output.
(127) As used herein, when an action is “based on” something, this means the action is based at least in part on at least a part of the something. As used herein, the terms “substantially,” “approximately” and “about” are defined as being largely but not necessarily wholly what is specified (and include wholly what is specified) as understood by one of ordinary skill in the art. In any disclosed embodiment, the term “substantially,” “approximately,” or “about” may be substituted with “within [a percentage] of” what is specified, where the percentage includes 0.1, 1, 5, and 10 percent.
(128) Illustrative Systems
(129) FIG. **6** depicts a simplified diagram of a distributed system **600**. In the illustrated example, distributed system **600** includes one or more client computing devices **602**, **604**, **606**, and **608**, coupled to a server **612** via one or more communication networks **610**. Clients computing devices **602**, **604**, **606**, and **608** may be configured to execute one or more applications.
(130) In various examples, server **612** may be adapted to run one or more services or software applications that enable one or more embodiments described in this disclosure. In certain examples, server **612** may also provide other services or software applications that may include non-virtual and virtual environments. In some examples, these services may be offered as web-based or cloud services, such as under a Software as a Service (SaaS) model to the users of client computing devices **602**, **604**, **606**, and/or **608**. Users operating client computing devices **602**, **604**, **606**, and/or **608** may in turn utilize one or more client applications to interact with server **612** to utilize the services provided by these components.
(131) In the configuration depicted in FIG. **6**, server **612** may include one or more components **618**, **620** and **622** that implement the functions performed by server **612**. These components may include

software components that may be executed by one or more processors, hardware components, or combinations thereof. It should be appreciated that various different system configurations are possible, which may be different from distributed system **600**. The example shown in FIG. **6** is thus one example of a distributed system for implementing an example system and is not intended to be limiting.

(132) Users may use client computing devices **602**, **604**, **606**, and/or **608** to execute one or more applications, models or chatbots, which may generate one or more events or models that may then be implemented or serviced in accordance with the teachings of this disclosure. A client device may provide an interface that enables a user of the client device to interact with the client device. The client device may also output information to the user via this interface. Although FIG. **6** depicts only four client computing devices, any number of client computing devices may be supported.

(133) The client devices may include various types of computing systems such as portable handheld devices, general purpose computers such as personal computers and laptops, workstation computers, wearable devices, gaming systems, thin clients, various messaging devices, sensors or other sensing devices, and/or the like. These computing devices may run various types and versions of software applications and operating systems (e.g., Microsoft Windows®, Apple Macintosh®, UNIX® or UNIX-like operating systems, Linux or Linux-like operating systems such as Google Chrome™ OS) including various mobile operating systems (e.g., Microsoft Windows Mobile®, iOS®, Windows Phone®, Android™, BlackBerry®, Palm OS®). Portable handheld devices may include cellular phones, smartphones, (e.g., an iPhone®), tablets (e.g., iPad®), personal digital assistants (PDAs), and the like. Wearable devices may include Google Glass® head mounted display, and other devices. Gaming systems may include various handheld gaming devices, Internet-enabled gaming devices (e.g., a Microsoft Xbox® gaming console with or without a Kinect® gesture input device, Sony PlayStation® system, various gaming systems provided by Nintendo®, and others), and the like. The client devices may be capable of executing various different applications such as various Internet-related apps, communication applications (e.g., E-mail applications, short message service (SMS) applications) and may use various communication protocols.

(134) Network(s) **610** may be any type of network familiar to those skilled in the art that may support data communications using any of a variety of available protocols, including without limitation TCP/IP (transmission control protocol/Internet protocol), SNA (systems network architecture), IPX (Internet packet exchange), AppleTalk®, and the like. Merely by way of example, network(s) **610** may be a local area network (LAN), networks based on Ethernet, Token-Ring, a wide-area network (WAN), the Internet, a virtual network, a virtual private network (VPN), an intranet, an extranet, a public switched telephone network (PSTN), an infra-red network, a wireless network (e.g., a network operating under any of the Institute of Electrical and Electronics (IEEE) 1002.11 suite of protocols, Bluetooth®, and/or any other wireless protocol), and/or any combination of these and/or other networks.

(135) Server **612** may be composed of one or more general purpose computers, specialized server computers (including, by way of example, PC (personal computer) servers, UNIX® servers, mid-range servers, mainframe computers, rack-mounted servers, etc.), server farms, server clusters, or any other appropriate arrangement and/or combination. Server **612** may include one or more virtual machines running virtual operating systems, or other computing architectures involving virtualization such as one or more flexible pools of logical storage devices that may be virtualized to maintain virtual storage devices for the server. In various examples, server **612** may be adapted to run one or more services or software applications that provide the functionality described in the foregoing disclosure.

(136) The computing systems in server **612** may run one or more operating systems including any of those discussed above, as well as any commercially available server operating system. Server **612** may also run any of a variety of additional server applications and/or mid-tier applications,

including HTTP (hypertext transport protocol) servers, FTP (file transfer protocol) servers, CGI (common gateway interface) servers, JAVA® servers, database servers, and the like. Exemplary database servers include without limitation those commercially available from Oracle®, Microsoft®, Sybase®, IBM® (International Business Machines), and the like.

(137) In some implementations, server **612** may include one or more applications to analyze and consolidate data feeds and/or event updates received from users of client computing devices **602**, **604**, **606**, and **608**. As an example, data feeds and/or event updates may include, but are not limited to, Twitter® feeds, Facebook® updates or real-time updates received from one or more third party information sources and continuous data streams, which may include real-time events related to sensor data applications, financial tickers, network performance measuring tools (e.g., network monitoring and traffic management applications), clickstream analysis tools, automobile traffic monitoring, and the like. Server **612** may also include one or more applications to display the data feeds and/or real-time events via one or more display devices of client computing devices **602**, **604**, **606**, and **608**.

(138) Distributed system **600** may also include one or more data repositories **614**, **616**. These data repositories may be used to store data and other information in certain examples. For example, one or more of the data repositories **614**, **616** may be used to store information such as information related to chatbot performance or generated models for use by chatbots used by server **612** when performing various functions in accordance with various embodiments. Data repositories **614**, **616** may reside in a variety of locations. For example, a data repository used by server **612** may be local to server **612** or may be remote from server **612** and in communication with server **612** via a network-based or dedicated connection. Data repositories **614**, **616** may be of different types. In certain examples, a data repository used by server **612** may be a database, for example, a relational database, such as databases provided by Oracle Corporation® and other vendors. One or more of these databases may be adapted to enable storage, update, and retrieval of data to and from the database in response to SQL-formatted commands.

(139) In certain examples, one or more of data repositories **614**, **616** may also be used by applications to store application data. The data repositories used by applications may be of different types such as, for example, a key-value store repository, an object store repository, or a general storage repository supported by a file system.

(140) In certain examples, the functionalities described in this disclosure may be offered as services via a cloud environment. FIG. **7** is a simplified block diagram of a cloud-based system environment **700** in which various services may be offered as cloud services in accordance with certain examples. In the example depicted in FIG. **7**, cloud infrastructure system **702** may provide one or more cloud services that may be requested by users using one or more client computing devices **704**, **706**, and **708**. Cloud infrastructure system **702** may comprise one or more computers and/or servers that may include those described above for server **612**. The computers in cloud infrastructure system **702** may be organized as general-purpose computers, specialized server computers, server farms, server clusters, or any other appropriate arrangement and/or combination.

(141) Network(s) **710** may facilitate communication and exchange of data between clients **704**, **706**, and **708** and cloud infrastructure system **702**. Network(s) **710** may include one or more networks. The networks may be of the same or different types. Network(s) **710** may support one or more communication protocols, including wired and/or wireless protocols, for facilitating the communications.

(142) The example depicted in FIG. **7** is only one example of a cloud infrastructure system and is not intended to be limiting. It should be appreciated that, in some other examples, cloud infrastructure system **702** may have more or fewer components than those depicted in FIG. **7**, may combine two or more components, or may have a different configuration or arrangement of components. For example, although FIG. **7** depicts three client computing devices, any number of client computing devices may be supported in alternative examples.

(143) The term cloud service is generally used to refer to a service that is made available to users on demand and via a communication network such as the Internet by systems (e.g., cloud infrastructure system **702**) of a service provider. Typically, in a public cloud environment, servers and systems that make up the cloud service provider's system are different from the customer's own on-premise servers and systems. The cloud service provider's systems are managed by the cloud service provider. Customers may thus avail themselves of cloud services provided by a cloud service provider without having to purchase separate licenses, support, or hardware and software resources for the services. For example, a cloud service provider's system may host an application, and a user may, via the Internet, on demand, order and use the application without the user having to buy infrastructure resources for executing the application. Cloud services are designed to provide easy, scalable access to applications, resources and services. Several providers offer cloud services. For example, several cloud services are offered by Oracle Corporation® of Redwood Shores, California, such as middleware services, database services, Java cloud services, and others.
(144) In certain examples, cloud infrastructure system **702** may provide one or more cloud services using different models such as under a Software as a Service (SaaS) model, a Platform as a Service (PaaS) model, an Infrastructure as a Service (IaaS) model, and others, including hybrid service models. Cloud infrastructure system **702** may include a suite of applications, middleware, databases, and other resources that enable provision of the various cloud services.
(145) A SaaS model enables an application or software to be delivered to a customer over a communication network like the Internet, as a service, without the customer having to buy the hardware or software for the underlying application. For example, a SaaS model may be used to provide customers access to on-demand applications that are hosted by cloud infrastructure system **702**. Examples of SaaS services provided by Oracle Corporation® include, without limitation, various services for human resources/capital management, customer relationship management (CRM), enterprise resource planning (ERP), supply chain management (SCM), enterprise performance management (EPM), analytics services, social applications, and others.
(146) An IaaS model is generally used to provide infrastructure resources (e.g., servers, storage, hardware and networking resources) to a customer as a cloud service to provide elastic compute and storage capabilities. Various IaaS services are provided by Oracle Corporation®.
(147) A PaaS model is generally used to provide, as a service, platform and environment resources that enable customers to develop, run, and manage applications and services without the customer having to procure, build, or maintain such resources. Examples of PaaS services provided by Oracle Corporation® include, without limitation, Oracle Java Cloud Service (JCS), Oracle Database Cloud Service (DBCS), data management cloud service, various application development solutions services, and others.
(148) Cloud services are generally provided on an on-demand self-service basis, subscription-based, elastically scalable, reliable, highly available, and secure manner. For example, a customer, via a subscription order, may order one or more services provided by cloud infrastructure system **702**. Cloud infrastructure system **702** then performs processing to provide the services requested in the customer's subscription order. For example, a user may use utterances to request the cloud infrastructure system to take a certain action (e.g., an intent), as described above, and/or provide services for a chatbot system as described herein. Cloud infrastructure system **702** may be configured to provide one or even multiple cloud services.
(149) Cloud infrastructure system **702** may provide the cloud services via different deployment models. In a public cloud model, cloud infrastructure system **702** may be owned by a third party cloud services provider and the cloud services are offered to any general public customer, where the customer may be an individual or an enterprise. In certain other examples, under a private cloud model, cloud infrastructure system **702** may be operated within an organization (e.g., within an enterprise organization) and services provided to customers that are within the organization. For example, the customers may be various departments of an enterprise such as the Human Resources

department, the Payroll department, etc. or even individuals within the enterprise. In certain other examples, under a community cloud model, the cloud infrastructure system **702** and the services provided may be shared by several organizations in a related community. Various other models such as hybrids of the above mentioned models may also be used.

(150) Client computing devices **704**, **706**, and **708** may be of different types (such as client computing devices **602**, **604**, **606**, and **608** depicted in FIG. **6**) and may be capable of operating one or more client applications. A user may use a client device to interact with cloud infrastructure system **702**, such as to request a service provided by cloud infrastructure system **702**. For example, a user may use a client device to request information or action from a chatbot as described in this disclosure.

(151) In some examples, the processing performed by cloud infrastructure system **702** for providing services may involve model training and deployment. This analysis may involve using, analyzing, and manipulating data sets to train and deploy one or more models. This analysis may be performed by one or more processors, possibly processing the data in parallel, performing simulations using the data, and the like. For example, big data analysis may be performed by cloud infrastructure system **702** for generating and training one or more models for a chatbot system. The data used for this analysis may include structured data (e.g., data stored in a database or structured according to a structured model) and/or unstructured data (e.g., data blobs (binary large objects)).

(152) As depicted in the example in FIG. **7**, cloud infrastructure system **702** may include infrastructure resources **730** that are utilized for facilitating the provision of various cloud services offered by cloud infrastructure system **702**. Infrastructure resources **730** may include, for example, processing resources, storage or memory resources, networking resources, and the like. In certain examples, the storage virtual machines that are available for servicing storage requested from applications may be part of cloud infrastructure system **702**. In other examples, the storage virtual machines may be part of different systems.

(153) In certain examples, to facilitate efficient provisioning of these resources for supporting the various cloud services provided by cloud infrastructure system **702** for different customers, the resources may be bundled into sets of resources or resource modules (also referred to as “pods”). Each resource module or pod may comprise a pre-integrated and optimized combination of resources of one or more types. In certain examples, different pods may be pre-provisioned for different types of cloud services. For example, a first set of pods may be provisioned for a database service, a second set of pods, which may include a different combination of resources than a pod in the first set of pods, may be provisioned for Java service, and the like. For some services, the resources allocated for provisioning the services may be shared between the services.

(154) Cloud infrastructure system **702** may itself internally use services **732** that are shared by different components of cloud infrastructure system **702** and which facilitate the provisioning of services by cloud infrastructure system **702**. These internal shared services may include, without limitation, a security and identity service, an integration service, an enterprise repository service, an enterprise manager service, a virus scanning and whitelist service, a high availability, backup and recovery service, service for enabling cloud support, an email service, a notification service, a file transfer service, and the like.

(155) Cloud infrastructure system **702** may comprise multiple subsystems. These subsystems may be implemented in software, or hardware, or combinations thereof. As depicted in FIG. **7**, the subsystems may include a user interface subsystem **712** that enables users or customers of cloud infrastructure system **702** to interact with cloud infrastructure system **702**. User interface subsystem **712** may include various different interfaces such as a web interface **714**, an online store interface **716** where cloud services provided by cloud infrastructure system **702** are advertised and are purchasable by a consumer, and other interfaces **718**. For example, a customer may, using a client device, request (service request **734**) one or more services provided by cloud infrastructure system **702** using one or more of interfaces **714**, **716**, and **718**. For example, a customer may access the

online store, browse cloud services offered by cloud infrastructure system **702**, and place a subscription order for one or more services offered by cloud infrastructure system **702** that the customer wishes to subscribe to. The service request may include information identifying the customer and one or more services that the customer desires to subscribe to. For example, a customer may place a subscription order for a service offered by cloud infrastructure system **702**. As part of the order, the customer may provide information identifying a chatbot system for which the service is to be provided and optionally one or more credentials for the chatbot system.

(156) In certain examples, such as the example depicted in FIG. **7**, cloud infrastructure system **702** may comprise an order management subsystem (OMS) **720** that is configured to process the new order. As part of this processing, OMS **720** may be configured to: create an account for the customer, if not done already; receive billing and/or accounting information from the customer that is to be used for billing the customer for providing the requested service to the customer; verify the customer information; upon verification, book the order for the customer; and orchestrate various workflows to prepare the order for provisioning.

(157) Once properly validated, OMS **720** may then invoke the order provisioning subsystem (OPS) **724** that is configured to provision resources for the order including processing, memory, and networking resources. The provisioning may include allocating resources for the order and configuring the resources to facilitate the service requested by the customer order. The manner in which resources are provisioned for an order and the type of the provisioned resources may depend upon the type of cloud service that has been ordered by the customer. For example, according to one workflow, OPS **724** may be configured to determine the particular cloud service being requested and identify a number of pods that may have been pre-configured for that particular cloud service. The number of pods that are allocated for an order may depend upon the size/amount/level/scope of the requested service. For example, the number of pods to be allocated may be determined based upon the number of users to be supported by the service, the duration of time for which the service is being requested, and the like. The allocated pods may then be customized for the particular requesting customer for providing the requested service.

(158) In certain examples, setup phase processing, as described above, may be performed by cloud infrastructure system **702** as part of the provisioning process. Cloud infrastructure system **702** may generate an application ID and select a storage virtual machine for an application from among storage virtual machines provided by cloud infrastructure system **702** itself or from storage virtual machines provided by other systems other than cloud infrastructure system **702**.

(159) Cloud infrastructure system **702** may send a response or notification **744** to the requesting customer to indicate when the requested service is now ready for use. In some instances, information (e.g., a link) may be sent to the customer that enables the customer to start using and availing the benefits of the requested services. In certain examples, for a customer requesting the service, the response may include a chatbot system ID generated by cloud infrastructure system **702** and information identifying a chatbot system selected by cloud infrastructure system **702** for the chatbot system corresponding to the chatbot system ID.

(160) Cloud infrastructure system **702** may provide services to multiple customers. For each customer, cloud infrastructure system **702** is responsible for managing information related to one or more subscription orders received from the customer, maintaining customer data related to the orders, and providing the requested services to the customer. Cloud infrastructure system **702** may also collect usage statistics regarding a customer's use of subscribed services. For example, statistics may be collected for the amount of storage used, the amount of data transferred, the number of users, and the amount of system up time and system down time, and the like. This usage information may be used to bill the customer. Billing may be done, for example, on a monthly cycle.

(161) Cloud infrastructure system **702** may provide services to multiple customers in parallel. Cloud infrastructure system **702** may store information for these customers, including possibly

proprietary information. In certain examples, cloud infrastructure system **702** comprises an identity management subsystem (IMS) **728** that is configured to manage customer information and provide the separation of the managed information such that information related to one customer is not accessible by another customer. IMS **728** may be configured to provide various security-related services such as identity services, such as information access management, authentication and authorization services, services for managing customer identities and roles and related capabilities, and the like.

(162) FIG. **8** illustrates an example of computer system **800**. In some examples, computer system **800** may be used to implement any of the digital assistant or chatbot systems within a distributed environment, and various servers and computer systems described above. As shown in FIG. **8**, computer system **800** includes various subsystems including a processing subsystem **804** that communicates with a number of other subsystems via a bus subsystem **802**. These other subsystems may include a processing acceleration unit **806**, an I/O subsystem **808**, a storage subsystem **818**, and a communications subsystem **824**. Storage subsystem **818** may include non-transitory computer-readable storage media including storage media **822** and a system memory **810**.

(163) Bus subsystem **802** provides a mechanism for letting the various components and subsystems of computer system **800** communicate with each other as intended. Although bus subsystem **802** is shown schematically as a single bus, alternative examples of the bus subsystem may utilize multiple buses. Bus subsystem **802** may be any of several types of bus structures including a memory bus or memory controller, a peripheral bus, a local bus using any of a variety of bus architectures, and the like. For example, such architectures may include an Industry Standard Architecture (ISA) bus, Micro Channel Architecture (MCA) bus, Enhanced ISA (EISA) bus, Video Electronics Standards Association (VESA) local bus, and Peripheral Component Interconnect (PCI) bus, which may be implemented as a Mezzanine bus manufactured to the IEEE P1386.1 standard, and the like.

(164) Processing subsystem **804** controls the operation of computer system **800** and may comprise one or more processors, application specific integrated circuits (ASICs), or field programmable gate arrays (FPGAs). The processors may include be single core or multicore processors. The processing resources of computer system **800** may be organized into one or more processing units **832**, **834**, etc. A processing unit may include one or more processors, one or more cores from the same or different processors, a combination of cores and processors, or other combinations of cores and processors. In some examples, processing subsystem **804** may include one or more special purpose co-processors such as graphics processors, digital signal processors (DSPs), or the like. In some examples, some or all of the processing units of processing subsystem **804** may be implemented using customized circuits, such as application specific integrated circuits (ASICs), or field programmable gate arrays (FPGAs).

(165) In some examples, the processing units in processing subsystem **804** may execute instructions stored in system memory **810** or on computer readable storage media **822**. In various examples, the processing units may execute a variety of programs or code instructions and may maintain multiple concurrently executing programs or processes. At any given time, some or all of the program code to be executed may be resident in system memory **810** and/or on computer-readable storage media **822** including potentially on one or more storage devices. Through suitable programming, processing subsystem **804** may provide various functionalities described above. In instances where computer system **800** is executing one or more virtual machines, one or more processing units may be allocated to each virtual machine.

(166) In certain examples, a processing acceleration unit **806** may optionally be provided for performing customized processing or for off-loading some of the processing performed by processing subsystem **804** so as to accelerate the overall processing performed by computer system **800**.

(167) I/O subsystem **808** may include devices and mechanisms for inputting information to computer system **800** and/or for outputting information from or via computer system **800**. In general, use of the term input device is intended to include all possible types of devices and mechanisms for inputting information to computer system **800**. User interface input devices may include, for example, a keyboard, pointing devices such as a mouse or trackball, a touchpad or touch screen incorporated into a display, a scroll wheel, a click wheel, a dial, a button, a switch, a keypad, audio input devices with voice command recognition systems, microphones, and other types of input devices. User interface input devices may also include motion sensing and/or gesture recognition devices such as the Microsoft Kinect® motion sensor that enables users to control and interact with an input device, the Microsoft Xbox® 360 game controller, devices that provide an interface for receiving input using gestures and spoken commands. User interface input devices may also include eye gesture recognition devices such as the Google Glass® blink detector that detects eye activity (e.g., "blinking" while taking pictures and/or making a menu selection) from users and transforms the eye gestures as inputs to an input device (e.g., Google Glass®). Additionally, user interface input devices may include voice recognition sensing devices that enable users to interact with voice recognition systems (e.g., Siri® navigator) through voice commands.
(168) Other examples of user interface input devices include, without limitation, three dimensional (3D) mice, joysticks or pointing sticks, gamepads and graphic tablets, and audio/visual devices such as speakers, digital cameras, digital camcorders, portable media players, webcams, image scanners, fingerprint scanners, barcode reader 3D scanners, 3D printers, laser rangefinders, and eye gaze tracking devices. Additionally, user interface input devices may include, for example, medical imaging input devices such as computed tomography, magnetic resonance imaging, position emission tomography, and medical ultrasonography devices. User interface input devices may also include, for example, audio input devices such as MIDI keyboards, digital musical instruments and the like.
(169) In general, use of the term output device is intended to include all possible types of devices and mechanisms for outputting information from computer system **800** to a user or other computer. User interface output devices may include a display subsystem, indicator lights, or non-visual displays such as audio output devices, etc. The display subsystem may be a cathode ray tube (CRT), a flat-panel device, such as that using a liquid crystal display (LCD) or plasma display, a projection device, a touch screen, and the like. For example, user interface output devices may include, without limitation, a variety of display devices that visually convey text, graphics and audio/video information such as monitors, printers, speakers, headphones, automotive navigation systems, plotters, voice output devices, and modems.
(170) Storage subsystem **818** provides a repository or data store for storing information and data that is used by computer system **800**. Storage subsystem **818** provides a tangible non-transitory computer-readable storage medium for storing the basic programming and data constructs that provide the functionality of some examples. Storage subsystem **818** may store software (e.g., programs, code modules, instructions) that when executed by processing subsystem **804** provides the functionality described above. The software may be executed by one or more processing units of processing subsystem **804**. Storage subsystem **818** may also provide authentication in accordance with the teachings of this disclosure.
(171) Storage subsystem **818** may include one or more non-transitory memory devices, including volatile and non-volatile memory devices. As shown in FIG. **8**, storage subsystem **818** includes a system memory **810** and a computer-readable storage media **822**. System memory **810** may include a number of memories including a volatile main random-access memory (RAM) for storage of instructions and data during program execution and a non-volatile read only memory (ROM) or flash memory in which fixed instructions are stored. In some implementations, a basic input/output system (BIOS), containing the basic routines that help to transfer information between elements

within computer system **800**, such as during start-up, may typically be stored in the ROM. The RAM typically contains data and/or program modules that are presently being operated and executed by processing subsystem **804**. In some implementations, system memory **810** may include multiple different types of memory, such as static random-access memory (SRAM), dynamic random-access memory (DRAM), and the like.

(172) Byway of example, and not limitation, as depicted in FIG. **8**, system memory **810** may load application programs **812** that are being executed, which may include various applications such as Web browsers, mid-tier applications, relational database management systems (RDBMS), etc., program data **814**, and an operating system **816**. By way of example, operating system **816** may include various versions of Microsoft Windows®, Apple Macintosh®, and/or Linux operating systems, a variety of commercially-available UNIX® or UNIX-like operating systems (including without limitation the variety of GNU/Linux operating systems, the Google Chrome® OS, and the like) and/or mobile operating systems such as iOS, Windows® Phone, Android® OS, BlackBerry® OS, Palm® OS operating systems, and others.

(173) Computer-readable storage media **822** may store programming and data constructs that provide the functionality of some examples. Computer-readable media **822** may provide storage of computer-readable instructions, data structures, program modules, and other data for computer system **800**. Software (programs, code modules, instructions) that, when executed by processing subsystem **804** provides the functionality described above, may be stored in storage subsystem **818**. By way of example, computer-readable storage media **822** may include non-volatile memory such as a hard disk drive, a magnetic disk drive, an optical disk drive such as a CD ROM, DVD, a Blu-Ray® disk, or other optical media. Computer-readable storage media **822** may include, but is not limited to, Zip® drives, flash memory cards, universal serial bus (USB) flash drives, secure digital (SD) cards, DVD disks, digital video tape, and the like. Computer-readable storage media **822** may also include, solid-state drives (SSD) based on non-volatile memory such as flash-memory based SSDs, enterprise flash drives, solid state ROM, and the like, SSDs based on volatile memory such as solid-state RAM, dynamic RAM, static RAM, DRAM-based SSDs, magneto resistive RAM (MRAM) SSDs, and hybrid SSDs that use a combination of DRAM and flash memory based SSDs.

(174) In certain examples, storage subsystem **818** may also include a computer-readable storage media reader **820** that may further be connected to computer-readable storage media **822**. Reader **820** may receive and be configured to read data from a memory device such as a disk, a flash drive, etc.

(175) In certain examples, computer system **800** may support virtualization technologies, including but not limited to virtualization of processing and memory resources. For example, computer system **800** may provide support for executing one or more virtual machines. In certain examples, computer system **800** may execute a program such as a hypervisor that facilitated the configuring and managing of the virtual machines. Each virtual machine may be allocated memory, compute (e.g., processors, cores), I/O, and networking resources. Each virtual machine generally runs independently of the other virtual machines. A virtual machine typically runs its own operating system, which may be the same as or different from the operating systems executed by other virtual machines executed by computer system **800**. Accordingly, multiple operating systems may potentially be run concurrently by computer system **800**.

(176) Communications subsystem **824** provides an interface to other computer systems and networks. Communications subsystem **824** serves as an interface for receiving data from and transmitting data to other systems from computer system **800**. For example, communications subsystem **824** may enable computer system **800** to establish a communication channel to one or more client devices via the Internet for receiving and sending information from and to the client devices. For example, when computer system **800** is used to implement bot system **120** depicted in FIG. **1**, the communication subsystem may be used to communicate with a chatbot system selected for an application.

(177) Communication subsystem **824** may support both wired and/or wireless communication protocols. In certain examples, communications subsystem **824** may include radio frequency (RF) transceiver components for accessing wireless voice and/or data networks (e.g., using cellular telephone technology, advanced data network technology, such as 3G, 4G or EDGE (enhanced data rates for global evolution), Wi-Fi (IEEE 802.XX family standards, or other mobile communication technologies, or any combination thereof), global positioning system (GPS) receiver components, and/or other components. In some examples, communications subsystem **824** may provide wired network connectivity (e.g., Ethernet) in addition to or instead of a wireless interface.
(178) Communication subsystem **824** may receive and transmit data in various forms. In some examples, in addition to other forms, communications subsystem **824** may receive input communications in the form of structured and/or unstructured data feeds **826**, event streams **828**, event updates **830**, and the like. For example, communications subsystem **824** may be configured to receive (or send) data feeds **826** in real-time from users of social media networks and/or other communication services such as Twitter® feeds, Facebook® updates, web feeds such as Rich Site Summary (RSS) feeds, and/or real-time updates from one or more third party information sources.
(179) In certain examples, communications subsystem **824** may be configured to receive data in the form of continuous data streams, which may include event streams **828** of real-time events and/or event updates **830**, that may be continuous or unbounded in nature with no explicit end. Examples of applications that generate continuous data may include, for example, sensor data applications, financial tickers, network performance measuring tools (e.g., network monitoring and traffic management applications), clickstream analysis tools, automobile traffic monitoring, and the like.
(180) Communications subsystem **824** may also be configured to communicate data from computer system **800** to other computer systems or networks. The data may be communicated in various different forms such as structured and/or unstructured data feeds **826**, event streams **828**, event updates **830**, and the like to one or more databases that may be in communication with one or more streaming data source computers coupled to computer system **800**.
(181) Computer system **800** may be one of various types, including a handheld portable device (e.g., an iPhone® cellular phone, an iPad® computing tablet, a PDA), a wearable device (e.g., a Google Glass® head mounted display), a personal computer, a workstation, a mainframe, a kiosk, a server rack, or any other data processing system. Due to the ever-changing nature of computers and networks, the description of computer system **800** depicted in FIG. **8** is intended only as a specific example. Many other configurations having more or fewer components than the system depicted in FIG. **8** are possible. Based on the disclosure and teachings provided herein, it should be appreciated there are other ways and/or methods to implement the various examples.
(182) Although specific examples have been described, various modifications, alterations, alternative constructions, and equivalents are possible. Examples are not restricted to operation within certain specific data processing environments but are free to operate within a plurality of data processing environments. Additionally, although certain examples have been described using a particular series of transactions and steps, it should be apparent to those skilled in the art that this is not intended to be limiting. Although some flowcharts describe operations as a sequential process, many of the operations may be performed in parallel or concurrently. In addition, the order of the operations may be rearranged. A process may have additional steps not included in the figure. Various features and aspects of the above-described examples may be used individually or jointly.
(183) Further, while certain examples have been described using a particular combination of hardware and software, it should be recognized that other combinations of hardware and software are also possible. Certain examples may be implemented only in hardware, or only in software, or using combinations thereof. The various processes described herein may be implemented on the same processor or different processors in any combination.
(184) Where devices, systems, components or modules are described as being configured to perform certain operations or functions, such configuration may be accomplished, for example, by

designing electronic circuits to perform the operation, by programming programmable electronic circuits (such as microprocessors) to perform the operation such as by executing computer instructions or code, or processors or cores programmed to execute code or instructions stored on a non-transitory memory medium, or any combination thereof. Processes may communicate using a variety of techniques including but not limited to conventional techniques for inter-process communications, and different pairs of processes may use different techniques, or the same pair of processes may use different techniques at different times.

(185) Specific details are given in this disclosure to provide a thorough understanding of the examples. However, examples may be practiced without these specific details. For example, well-known circuits, processes, algorithms, structures, and techniques have been shown without unnecessary detail in order to avoid obscuring the examples. This description provides example examples only, and is not intended to limit the scope, applicability, or configuration of other examples. Rather, the preceding description of the examples will provide those skilled in the art with an enabling description for implementing various examples. Various changes may be made in the function and arrangement of elements.

(186) The specification and drawings are, accordingly, to be regarded in an illustrative rather than a restrictive sense. It will, however, be evident that additions, subtractions, deletions, and other modifications and changes may be made thereunto without departing from the broader spirit and scope as set forth in the claims. Thus, although specific examples have been described, these are not intended to be limiting. Various modifications and equivalents are within the scope of the following claims.

(187) In the foregoing specification, aspects of the disclosure are described with reference to specific examples thereof, but those skilled in the art will recognize that the disclosure is not limited thereto. Various features and aspects of the above-described disclosure may be used individually or jointly. Further, examples may be utilized in any number of environments and applications beyond those described herein without departing from the broader spirit and scope of the specification. The specification and drawings are, accordingly, to be regarded as illustrative rather than restrictive.

(188) In the foregoing description, for the purposes of illustration, methods were described in a particular order. It should be appreciated that in alternate examples, the methods may be performed in a different order than that described. It should also be appreciated that the methods described above may be performed by hardware components or may be embodied in sequences of machine-executable instructions, which may be used to cause a machine, such as a general-purpose or special-purpose processor or logic circuits programmed with the instructions to perform the methods. These machine-executable instructions may be stored on one or more machine readable mediums, such as CD-ROMs or other type of optical disks, floppy diskettes, ROMs, RAMs, EPROMs, EEPROMs, magnetic or optical cards, flash memory, or other types of machine-readable mediums suitable for storing electronic instructions. Alternatively, the methods may be performed by a combination of hardware and software.

(189) Where components are described as being configured to perform certain operations, such configuration may be accomplished, for example, by designing electronic circuits or other hardware to perform the operation, by programming programmable electronic circuits (e.g., microprocessors, or other suitable electronic circuits) to perform the operation, or any combination thereof.

(190) While illustrative examples of the application have been described in detail herein, it is to be understood that the inventive concepts may be otherwise variously embodied and employed, and that the appended claims are intended to be construed to include such variations, except as limited by the prior art.

Claims

1. A computer-implemented method comprising: accessing an utterance; classifying one or more named entities within the utterance into predefined classes, wherein the classifying results in each of the one or more named entities being assigned a class from the predefined classes, and the predefined classes match table names or attribute names within a database schema; searching one or more value lists within the database schema using one or more tokens from the utterance, wherein the searching identifies and outputs one or more value matches comprising: (i) any value within the one or more value lists that matches a token from the utterance and (ii) any attribute associated with a matching value; generating a data structure, according to a predefined format for the data structure, by storing: (i) each of the one or more named entities and the assigned class for each of the one or more named entities, (ii) each of the value matches and the token matching each of the value matches, and (iii) the utterance, wherein the predefined format comprises: (i) the utterance in a first position; (ii) a named entity of the one or more named entities and a position indicator that indicates where in the utterance the named entity can be found in a second position; and (iii) a value match of the one or more value matches and a position indicator that indicates where in the utterance a word or token associated with the value match can be found in a third position; using a machine learning model to translate the utterance into a logical form based on the data structure; and executing the logical form as a query on a database associated with the database schema to retrieve a result for the query.
2. The computer-implemented method of claim 1, further comprising extracting information from the database schema for each of the one or more named entities based on the class assigned to each of the one or more named entities, wherein the information includes a predefined format for values within a table or attribute associated with the class via the matching table name or attribute name, and wherein the generating the data structure comprises formatting each of the one or more named entities in the predefined format associated with the class assigned to each of the one or more named entities.
3. The computer-implemented method of claim 1, wherein the classifying extracts matched offsets for each of the one or more named entities, which are a numerical beginning and an ending position of the each of the one or more named entities within the utterance, and wherein the generating the data structure further comprises organizing and storing the matched offsets for each of the one or more named entities in the predefined format for the data structure.
4. The computer-implemented method of claim 1, wherein the one or more value lists are structured within one or more indexes, and the search is performed by running queries on the one or more indexes using the tokens from the utterance.
5. The computer-implemented method of claim 1, wherein the searching extracts matched offsets for each of the tokens from the utterance, which are a numerical beginning and an ending position of the each of the tokens in the utterance, and wherein the generating the data structure further comprises organizing and storing the matched offsets for each of the tokens in the predefined format for the data structure.
6. The computer-implemented method of claim 1, wherein the one or more value lists comprise one or more attributes, values, and synonyms of the values, and the searching identifies and outputs value matches comprising: (i) any value within the one or more value lists that matches a token from the utterance, (ii) any value within the one or more value lists that has a synonym of the value that matches a token from the utterance, and (iii) any attribute associated with a matching value.
7. The computer-implemented method of claim 1, further comprising: generating a response to the utterance based on the result; and outputting, to a display or a user interface, the response.
8. A system comprising: one or more data processors; and one or more non-transitory computer-readable media storing instructions which, when executed by the one or more data processors,

cause the one or more data processors to perform operations comprising: accessing an utterance; classifying one or more named entities within the utterance into predefined classes, wherein the classifying results in each of the one or more named entities being assigned a class from the predefined classes, and the predefined classes match table names or attribute names within a database schema; searching one or more value lists within the database schema using one or more tokens from the utterance, wherein the searching identifies and outputs one or more value matches comprising: (i) any value within the one or more value lists that matches a token from the utterance and (ii) any attribute associated with a matching value; generating a data structure, according to a predefined format for the data structure, by storing: (i) each of the one or more named entities and the assigned class for each of the one or more named entities, (ii) each of the value matches and the token matching each of the value matches, and (iii) the utterance, wherein the predefined format comprises: (i) the utterance in a first position; (ii) a named entity of the one or more named entities and a position indicator that indicates where in the utterance the named entity can be found in a second position; and (iii) a value match of the one or more value matches and a position indicator that indicates where in the utterance a word or token associated with the value match can be found in a third position; using a machine learning model to translate the utterance into a logical form based on the data structure; and executing the logical form as a query on a database associated with the database schema to retrieve a result for the query.

9. The system of claim 8, wherein the operations further comprise extracting information from the database schema for each of the one or more named entities based on the class assigned to each of the one or more named entities, wherein the information includes a predefined format for values within a table or attribute associated with the class via the matching table name or attribute name, and wherein the generating the data structure comprises formatting each of the one or more named entities in the predefined format associated with the class assigned to each of the one or more named entities.

10. The system of claim 8, wherein the classifying extracts matched offsets for each of the one or more named entities, which are a numerical beginning and an ending position of the each of the one or more named entities within the utterance, and wherein the generating the data structure further comprises organizing and storing the matched offsets for each of the one or more named entities in the predefined format for the data structure.

11. The system of claim 8, wherein the one or more value lists are structured within one or more indexes, and the search is performed by running queries on the one or more indexes using the tokens from the utterance.

12. The system of claim 8, wherein the searching extracts matched offsets for each of the tokens from the utterance, which are a numerical beginning and an ending position of the each of the tokens in the utterance, and wherein the generating the data structure further comprises organizing and storing the matched offsets for each of the tokens in the predefined format for the data structure.

13. The system of claim 8, wherein the one or more value lists comprise one or more attributes, values, and synonyms of the values, and the searching identifies and outputs value matches comprising: (i) any value within the one or more value lists that matches a token from the utterance, (ii) any value within the one or more value lists that has a synonym of the value that matches a token from the utterance, and (iii) any attribute associated with a matching value.

14. The system of claim 8, wherein the operations further comprise: generating a response to the utterance based on the result; and outputting, to a display or a user interface, the response.

15. A computer-program product tangibly embodied in one or more non-transitory machine-readable media, including instructions configured to cause one or more data processors to perform the following operations: accessing an utterance; classifying one or more named entities within the utterance into predefined classes, wherein the classifying results in each of the one or more named entities being assigned a class from the predefined classes, and the predefined classes match table

names or attribute names within a database schema; searching one or more value lists within the database schema using one or more tokens from the utterance, wherein the searching identifies and outputs one or more value matches comprising: (i) any value within the one or more value lists that matches a token from the utterance and (ii) any attribute associated with a matching value; generating a data structure, according to a predefined format for the data structure, by storing: (i) each of the one or more named entities and the assigned class for each of the one or more named entities, (ii) each of the value matches and the token matching each of the value matches, and (iii) the utterance, wherein the predefined format comprises: (i) the utterance in a first position; (ii) a named entity of the one or more named entities and a position indicator that indicates where in the utterance the named entity can be found in a second position; and (iii) a value match of the one or more value matches and a position indicator that indicates where in the utterance a word or token associated with the value match can be found in a third position; using a machine learning model to translate the utterance into a logical form based on the data structure; and executing the logical form as a query on a database associated with the database schema to retrieve a result for the query.

16. The computer-program product of claim 15, wherein the operations further comprise extracting information from the database schema for each of the one or more named entities based on the class assigned to each of the one or more named entities, wherein the information includes a predefined format for values within a table or attribute associated with the class via the matching table name or attribute name, and wherein the generating the data structure comprises formatting each of the one or more named entities in the predefined format associated with the class assigned to each of the one or more named entities.

17. The computer-program product of claim 15, wherein the classifying extracts matched offsets for each of the one or more named entities, which are a numerical beginning and an ending position of the each of the one or more named entities within the utterance, and wherein the generating the data structure further comprises organizing and storing the matched offsets for each of the one or more named entities in the predefined format for the data structure.

18. The computer-program product of claim 15, wherein the one or more value lists are structured within one or more indexes, and the search is performed by running queries on the one or more indexes using the tokens from the utterance.

19. The computer-program product of claim 15, wherein the searching extracts matched offsets for each of the tokens from the utterance, which are a numerical beginning and an ending position of the each of the tokens in the utterance, and wherein the generating the data structure further comprises organizing and storing the matched offsets for each of the tokens in the predefined format for the data structure.

20. The computer-program product of claim 15, wherein the one or more value lists comprise one or more attributes, values, and synonyms of the values, and the searching identifies and outputs value matches comprising: (i) any value within the one or more value lists that matches a token from the utterance, (ii) any value within the one or more value lists that has a synonym of the value that matches a token from the utterance, and (iii) any attribute associated with a matching value.